

Exercises

0.1. In each of the following situations, indicate whether $f = O(g)$, or $f = \Omega(g)$, or both (in which case $f = \Theta(g)$).

$f(n)$	$g(n)$
(a) $n - 100$	$n - 200$
(b) $n^{1/2}$	$n^{2/3}$
(c) $100n + \log n$	$n + (\log n)^2$
(d) $n \log n$	$10n \log 10n$
(e) $\log 2n$	$\log 3n$
(f) $10 \log n$	$\log(n^2)$
(g) $n^{1.01}$	$n \log^2 n$
(h) $n^2 / \log n$	$n(\log n)^2$
(i) $n^{0.1}$	$(\log n)^{10}$
(j) $(\log n)^{\log n}$	$n / \log n$
(k) $\sqrt{n}$	$(\log n)^3$
(l) $n^{1/2}$	$5^{\log_2 n}$
(m) $n2^n$	3^n
(n) 2^n	2^{n+1}
(o) $n!$	2^n
(p) $(\log n)^{\log n}$	$2^{(\log_2 n)^2}$
(q) $\sum_{i=1}^n i^k$	n^{k+1}

0.2. Show that, if c is a positive real number, then $g(n) = 1 + c + c^2 + \dots + c^n$ is:

- (a) $\Theta(1)$ if $c < 1$.
- (b) $\Theta(n)$ if $c = 1$.
- (c) $\Theta(c^n)$ if $c > 1$.

The moral: in big- Θ terms, the sum of a geometric series is simply the first term if the series is strictly decreasing, the last term if the series is strictly increasing, or the number of terms if the series is unchanging.

0.3. The Fibonacci numbers $F_0, F_1, F_2, \dots$, are defined by the rule

$$F_0 = 0, F_1 = 1, F_n = F_{n-1} + F_{n-2}.$$

In this problem we will confirm that this sequence grows exponentially fast and obtain some bounds on its growth.

- (a) Use induction to prove that $F_n \geq 2^{0.5n}$ for $n \geq 6$.
- (b) Find a constant $c < 1$ such that $F_n \leq 2^{cn}$ for all $n \geq 0$. Show that your answer is correct.
- (c) What is the largest c you can find for which $F_n = \Omega(2^{cn})$?

0.4. Is there a faster way to compute the n th Fibonacci number than by fib2 (page 13)? One idea involves *matrices*.

We start by writing the equations $F_1 = F_1$ and $F_2 = F_0 + F_1$ in matrix notation:

$$\begin{pmatrix} F_1 \\ F_2 \end{pmatrix} = \begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix} \cdot \begin{pmatrix} F_0 \\ F_1 \end{pmatrix}.$$

Similarly,

$$\begin{pmatrix} F_2 \\ F_3 \end{pmatrix} = \begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix} \cdot \begin{pmatrix} F_1 \\ F_2 \end{pmatrix} = \begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix}^2 \cdot \begin{pmatrix} F_0 \\ F_1 \end{pmatrix}$$

and in general

$$\begin{pmatrix} F_n \\ F_{n+1} \end{pmatrix} = \begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix}^n \cdot \begin{pmatrix} F_0 \\ F_1 \end{pmatrix}.$$

So, in order to compute F_n , it suffices to raise this 2×2 matrix, call it X , to the n th power.

- (a) Show that two 2×2 matrices can be multiplied using 4 additions and 8 multiplications.

But how many matrix multiplications does it take to compute X^n ?

- (b) Show that $O(\log n)$ matrix multiplications suffice for computing X^n . (*Hint:* Think about computing X^8 .)

Thus the number of arithmetic operations needed by our matrix-based algorithm, call it `fib3`, is just $O(\log n)$, as compared to $O(n)$ for `fib2`. *Have we broken another exponential barrier?*

The catch is that our new algorithm involves multiplication, not just addition; and multiplications of large numbers are slower than additions. We have already seen that, when the complexity of arithmetic operations is taken into account, the running time of `fib2` becomes $O(n^2)$.

- (c) Show that all intermediate results of `fib3` are $O(n)$ bits long.
- (d) Let $M(n)$ be the running time of an algorithm for multiplying n -bit numbers, and assume that $M(n) = O(n^2)$ (the school method for multiplication, recalled in Chapter 1, achieves this). Prove that the running time of `fib3` is $O(M(n) \log n)$.
- (e) Can you prove that the running time of `fib3` is $O(M(n))$? (*Hint:* The lengths of the numbers being multiplied get doubled with every squaring.)

In conclusion, whether `fib3` is faster than `fib2` depends on whether we can multiply n -bit integers faster than $O(n^2)$. Do you think this is possible? (The answer is in Chapter 2.)

Finally, there is a formula for the Fibonacci numbers:

$$F_n = \frac{1}{\sqrt{5}} \left(\frac{1 + \sqrt{5}}{2} \right)^n - \frac{1}{\sqrt{5}} \left(\frac{1 - \sqrt{5}}{2} \right)^n.$$

So, it would appear that we only need to raise a couple of numbers to the n th power in order to compute F_n . The problem is that these numbers are irrational, and computing them to sufficient accuracy is nontrivial. In fact, our matrix method `fib3` can be seen as a roundabout way of raising these irrational numbers to the n th power. If you know your linear algebra, you should see why. (*Hint:* What are the eigenvalues of the matrix X ?)

Exercises

- 1.1. Show that in any base $b \geq 2$, the sum of any three single-digit numbers is at most two digits long.
- 1.2. Show that any binary integer is at most four times as long as the corresponding decimal integer. For very large numbers, what is the ratio of these two lengths, approximately?
- 1.3. A d -ary tree is a rooted tree in which each node has at most d children. Show that any d -ary tree with n nodes must have a depth of $\Omega(\log n / \log d)$. Can you give a precise formula for the minimum depth it could possibly have?
- 1.4. Show that

$$\log(n!) = \Theta(n \log n).$$

(Hint: To show an upper bound, compare $n!$ with n^n . To show a lower bound, compare it with $(n/2)^{n/2}$.)

- 1.5. Unlike a decreasing geometric series, the sum of the *harmonic series* $1, 1/2, 1/3, 1/4, 1/5, \dots$ diverges; that is,

$$\sum_{i=1}^{\infty} \frac{1}{i} = \infty.$$

It turns out that, for large n , the sum of the first n terms of this series can be well approximated as

$$\sum_{i=1}^n \frac{1}{i} \approx \ln n + \gamma,$$

where $\ln$ is natural logarithm (log base $e = 2.718\dots$) and γ is a particular constant $0.57721\dots$

Show that

$$\sum_{i=1}^n \frac{1}{i} = \Theta(\log n).$$

(Hint: To show an upper bound, decrease each denominator to the next power of two. For a lower bound, increase each denominator to the next power of 2.)

- 1.6. Prove that the grade-school multiplication algorithm (page 23), when applied to binary numbers, always gives the right answer.
- 1.7. How long does the recursive multiplication algorithm (page 24) take to multiply an n -bit number by an m -bit number? Justify your answer.
- 1.8. Justify the correctness of the recursive division algorithm given in page 25, and show that it takes time $O(n^2)$ on n -bit inputs.
- 1.9. Starting from the definition of $x \equiv y \pmod N$ (namely, that N divides $x - y$), justify the substitution rule

$$x \equiv x' \pmod N, y \equiv y' \pmod N \Rightarrow x + y \equiv x' + y' \pmod N,$$

and also the corresponding rule for multiplication.

- 1.10. Show that if $a \equiv b \pmod N$ and if M divides N then $a \equiv b \pmod M$.
- 1.11. Is $4^{1536} - 9^{4824}$ divisible by 35?
- 1.12. What is $2^{2^{2006}} \pmod 3$?
- 1.13. Is the difference of $5^{30,000}$ and $6^{123,456}$ a multiple of 31?
- 1.14. Suppose you want to compute the n th Fibonacci number F_n , modulo an integer p . Can you find an efficient way to do this? (Hint: Recall Exercise 0.4.)

- 1.15. Determine necessary and sufficient conditions on x and c so that the following holds: for any a, b , if $ax \equiv bx \pmod{c}$, then $a \equiv b \pmod{c}$.
- 1.16. The algorithm for computing $a^b \pmod{c}$ by repeated squaring does not necessarily lead to the minimum number of multiplications. Give an example of $b > 10$ where the exponentiation can be performed using fewer multiplications, by some other method.
- 1.17. Consider the problem of computing x^y for given integers x and y : we want the *whole* answer, not modulo a third integer. We know two algorithms for doing this: the iterative algorithm which performs $y - 1$ multiplications by x ; and the recursive algorithm based on the binary expansion of y .
Compare the time requirements of these two algorithms, assuming that the time to multiply an n -bit number by an m -bit number is $O(mn)$.
- 1.18. Compute $\gcd(210, 588)$ two different ways: by finding the factorization of each number, and by using Euclid's algorithm.
- 1.19. The *Fibonacci numbers* $F_0, F_1, \dots$ are given by the recurrence $F_{n+1} = F_n + F_{n-1}$, $F_0 = 0$, $F_1 = 1$. Show that for any $n \geq 1$, $\gcd(F_{n+1}, F_n) = 1$.
- 1.20. Find the inverse of: $20 \pmod{79}$, $3 \pmod{62}$, $21 \pmod{91}$, $5 \pmod{23}$.
- 1.21. How many integers modulo 11^3 have inverses? (Note: $11^3 = 1331$.)
- 1.22. Prove or disprove: If a has an inverse modulo b , then b has an inverse modulo a .
- 1.23. Show that if a has a multiplicative inverse modulo N , then this inverse is unique (modulo N).
- 1.24. If p is prime, how many elements of $\{0, 1, \dots, p^n - 1\}$ have an inverse modulo p^n ?
- 1.25. Calculate $2^{125} \pmod{127}$ using any method you choose. (*Hint*: 127 is prime.)
- 1.26. What is the least significant decimal digit of $17^{17^{17}}$? (*Hint*: For distinct primes p, q , and any $a \not\equiv 0 \pmod{pq}$, we proved the formula $a^{(p-1)(q-1)} \equiv 1 \pmod{pq}$ in Section 1.4.2.)
- 1.27. Consider an RSA key set with $p = 17$, $q = 23$, $N = 391$, and $e = 3$ (as in Figure 1.9). What value of d should be used for the secret key? What is the encryption of the message $M = 41$?
- 1.28. In an RSA cryptosystem, $p = 7$ and $q = 11$ (as in Figure 1.9). Find appropriate exponents d and e .
- 1.29. Let $[m]$ denote the set $\{0, 1, \dots, m - 1\}$. For each of the following families of hash functions, say whether or not it is universal, and determine how many random bits are needed to choose a function from the family.

- (a) $H = \{h_{a_1, a_2} : a_1, a_2 \in [m]\}$, where m is a fixed prime and

$$h_{a_1, a_2}(x_1, x_2) = a_1x_1 + a_2x_2 \pmod{m}.$$

Notice that each of these functions has signature $h_{a_1, a_2} : [m]^2 \rightarrow [m]$, that is, it maps a pair of integers in $[m]$ to a single integer in $[m]$.

- (b) H is as before, except that now $m = 2^k$ is some fixed power of 2.
- (c) H is the set of all functions $f : [m] \rightarrow [m - 1]$.
- 1.30. The grade-school algorithm for multiplying two n -bit binary numbers x and y consists of adding together n copies of x , each appropriately left-shifted. Each copy, when shifted, is at most $2n$ bits long.

In this problem, we will examine a scheme for adding n binary numbers, each m bits long, using a *circuit* or a *parallel architecture*. The main parameter of interest in this question is therefore

the depth of the circuit or the longest path from the input to the output of the circuit. This determines the total time taken for computing the function.

To add two m -bit binary numbers naively, we must wait for the carry bit from position $i - 1$ before we can figure out the i th bit of the answer. This leads to a circuit of depth $O(m)$. However carry lookahead circuits (see wikipedia.com if you want to know more about this) can add in $O(\log m)$ depth.

- (a) Assuming you have carry lookahead circuits for addition, show how to add n numbers each m bits long using a circuit of depth $O((\log n)(\log m))$.
- (b) When adding *three* m -bit binary numbers $x + y + z$, there is a trick we can use to parallelize the process. Instead of carrying out the addition completely, we can re-express the result as the sum of just *two* binary numbers $r + s$, such that the i th bits of r and s can be computed independently of the other bits. Show how this can be done. (*Hint*: One of the numbers represents carry bits.)
- (c) Show how to use the trick from the previous part to design a circuit of depth $O(\log n)$ for multiplying two n -bit numbers.

1.31. Consider the problem of computing $N! = 1 \cdot 2 \cdot 3 \cdots N$.

- (a) If N is an n -bit number, how many bits long is $N!$, approximately (in $\Theta(\cdot)$ form)?
- (b) Give an algorithm to compute $N!$ and analyze its running time.

1.32. A positive integer N is a *power* if it is of the form q^k , where q, k are positive integers and $k > 1$.

- (a) Give an efficient algorithm that takes as input a number N and determines whether it is a square, that is, whether it can be written as q^2 for some positive integer q . What is the running time of your algorithm?
- (b) Show that if $N = q^k$ (with N, q , and k all positive integers), then either $k \leq \log N$ or $N = 1$.
- (c) Give an efficient algorithm for determining whether a positive integer N is a power. Analyze its running time.

1.33. Give an efficient algorithm to compute the *least common multiple* of two n -bit numbers x and y , that is, the smallest number divisible by both x and y . What is the running time of your algorithm as a function of n ?

1.34. On page 37, we claimed that since about a $1/n$ fraction of n -bit numbers are prime, on average it is sufficient to draw $O(n)$ random n -bit numbers before hitting a prime. We now justify this rigorously.

Suppose a particular coin has a probability p of coming up heads. How many times must you toss it, on average, before it comes up heads? (*Hint*: Method 1: start by showing that the correct expression is $\sum_{i=1}^{\infty} i(1-p)^{i-1}p$. Method 2: if E is the average number of coin tosses, show that $E = 1 + (1-p)E$.)

1.35. Wilson's theorem says that a number N is prime if and only if

$$(N-1)! \equiv -1 \pmod{N}.$$

- (a) If p is prime, then we know every number $1 \leq x < p$ is invertible modulo p . Which of these numbers are their own inverse?
- (b) By pairing up multiplicative inverses, show that $(p-1)! \equiv -1 \pmod{p}$ for prime p .
- (c) Show that if N is *not* prime, then $(N-1)! \not\equiv -1 \pmod{N}$. (*Hint*: Consider $d = \gcd(N, (N-1)!)$.)

- (d) Unlike Fermat's Little theorem, Wilson's theorem is an if-and-only-if condition for primality. Why can't we immediately base a primality test on this rule?
- 1.36. *Square roots.* In this problem, we'll see that it is easy to compute square roots modulo a prime p with $p \equiv 3 \pmod{4}$.
- Suppose $p \equiv 3 \pmod{4}$. Show that $(p+1)/4$ is an integer.
 - We say x is a *square root* of a modulo p if $a \equiv x^2 \pmod{p}$. Show that if $p \equiv 3 \pmod{4}$ and if a has a square root modulo p , then $a^{(p+1)/4}$ is such a square root.
- 1.37. *The Chinese remainder theorem.*
- Make a table with three columns. The first column is all numbers from 0 to 14. The second is the residues of these numbers modulo 3; the third column is the residues modulo 5. What do you observe?
 - Prove that if p and q are distinct primes, then for every pair (j, k) with $0 \leq j < p$ and $0 \leq k < q$, there is a unique integer $0 \leq i < pq$ such that $i \equiv j \pmod{p}$ and $i \equiv k \pmod{q}$. (*Hint:* Prove that no two different i 's in this range can have the same (j, k) , and then count.)
 - In this one-to-one correspondence between integers and pairs, it is easy to go from i to (j, k) . Prove that the following formula takes you the other way:

$$i = \{j \cdot q \cdot (q^{-1} \pmod{p}) + k \cdot p \cdot (p^{-1} \pmod{q})\} \pmod{pq}.$$
 - Can you generalize parts (b) and (c) to more than two primes?
- 1.38. To see if a number, say 562437487, is divisible by 3, you just add up the digits of its decimal representation, and see if the result is divisible by 3. ($5 + 6 + 2 + 4 + 3 + 7 + 4 + 8 + 7 = 46$, so it is not divisible by 3.)
- To see if the same number is divisible by 11, you can do this: subdivide the number into pairs of digits, from the right-hand end (87, 74, 43, 62, 5), add these numbers, and see if the sum is divisible by 11 (if it's too big, repeat).
- How about 37? To see if the number is divisible by 37, subdivide it into triples from the end (487, 437, 562) add these up, and see if the sum is divisible by 37.
- This is true for any prime p other than 2 and 5. That is, for any prime $p \neq 2, 5$, there is an integer r such that in order to see if p divides a decimal number n , we break n into r -tuples of decimal digits (starting from the right-hand end), add up these r -tuples, and check if the sum is divisible by p .
- What is the smallest such r for $p = 13$? For $p = 17$?
 - Show that r is a divisor of $p - 1$.
- 1.39. Give a polynomial-time algorithm for computing $a^{b^c} \pmod{p}$, given a, b, c , and prime p .
- 1.40. Show that if x is a nontrivial square root of 1 modulo N , that is, if $x^2 \equiv 1 \pmod{N}$ but $x \not\equiv \pm 1 \pmod{N}$, then N must be composite. (For instance, $4^2 \equiv 1 \pmod{15}$ but $4 \not\equiv \pm 1 \pmod{15}$; thus 4 is a nontrivial square root of 1 modulo 15.)
- 1.41. *Quadratic residues.* Fix a positive integer N . We say that a is a *quadratic residue* modulo N if there exists x such that $a \equiv x^2 \pmod{N}$.
- Let N be an odd prime and a be a non-zero quadratic residue modulo N . Show that there are exactly two values in $\{0, 1, \dots, N-1\}$ satisfying $x^2 \equiv a \pmod{N}$.
 - Show that if N is an odd prime, there are exactly $(N+1)/2$ quadratic residues in $\{0, 1, \dots, N-1\}$.

- (c) Give an example of positive integers a and N such that $x^2 \equiv a \pmod{N}$ has more than two solutions in $\{0, 1, \dots, N-1\}$.
- 1.42. Suppose that instead of using a composite $N = pq$ in the RSA cryptosystem (Figure 1.9), we simply use a prime modulus p . As in RSA, we would have an encryption exponent e , and the encryption of a message $m \pmod{p}$ would be $m^e \pmod{p}$. Prove that this new cryptosystem is not secure, by giving an efficient algorithm to decrypt: that is, an algorithm that given p , e , and $m^e \pmod{p}$ as input, computes $m \pmod{p}$. Justify the correctness and analyze the running time of your decryption algorithm.
- 1.43. In the RSA cryptosystem, Alice's public key (N, e) is available to everyone. Suppose that her private key d is compromised and becomes known to Eve. Show that if $e = 3$ (a common choice) then Eve can efficiently factor N .
- 1.44. Alice and her three friends are all users of the RSA cryptosystem. Her friends have public keys $(N_i, e_i = 3)$, $i = 1, 2, 3$, where as always, $N_i = p_i q_i$ for randomly chosen n -bit primes p_i, q_i . Show that if Alice sends the same n -bit message M (encrypted using RSA) to each of her friends, then anyone who intercepts all three encrypted messages will be able to efficiently recover M .
(Hint: It helps to have solved problem 1.37 first.)
- 1.45. *RSA and digital signatures.* Recall that in the RSA public-key cryptosystem, each user has a public key $P = (N, e)$ and a secret key d . In a *digital signature scheme*, there are two algorithms, sign and verify. The sign procedure takes a message and a secret key, then outputs a signature σ . The verify procedure takes a public key (N, e) , a signature σ , and a message M , then returns "true" if σ could have been created by sign (when called with message M and the secret key corresponding to the public key (N, e)); "false" otherwise.
- Why would we want digital signatures?
 - An RSA signature consists of $\text{sign}(M, d) = M^d \pmod{N}$, where d is a secret key and N is part of the public key. Show that anyone who knows the public key (N, e) can perform $\text{verify}((N, e), M^d, M)$, i.e., they can check that a signature really was created by the private key. Give an implementation and prove its correctness.
 - Generate your own RSA modulus $N = pq$, public key e , and private key d (you don't need to use a computer). Pick p and q so you have a 4-digit modulus and work by hand. Now sign your name using the private exponent of this RSA modulus. To do this you will need to specify some one-to-one mapping from strings to integers in $[0, N-1]$. Specify any mapping you like. Give the mapping from your name to numbers $m_1, m_2, \dots, m_k$, then sign the first number by giving the value $m_1^d \pmod{N}$, and finally show that $(m_1^d)^e = m_1 \pmod{N}$.
 - Alice wants to write a message that looks like it was digitally signed by Bob. She notices that Bob's public RSA key is $(17, 391)$. To what exponent should she raise her message?
- 1.46. *Digital signatures, continued.* Consider the signature scheme of Exercise 1.45.
- Signing involves decryption, and is therefore risky. Show that if Bob agrees to sign anything he is asked to, Eve can take advantage of this and decrypt any message sent by Alice to Bob.
 - Suppose that Bob is more careful, and refuses to sign messages if their signatures look suspiciously like text. (We assume that a randomly chosen message—that is, a random number in the range $\{1, \dots, N-1\}$ —is very unlikely to look like text.) Describe a way in which Eve can nevertheless still decrypt messages from Alice to Bob, by getting Bob to sign messages whose signatures look random.

Exercises

- 2.1. Use the divide-and-conquer integer multiplication algorithm to multiply the two binary integers 10011011 and 10111010.
- 2.2. Show that for any positive integers n and any base b , there must some power of b lying in the range $[n, bn]$.
- 2.3. Section 2.2 describes a method for solving recurrence relations which is based on analyzing the recursion tree and deriving a formula for the work done at each level. Another (closely related) method is to expand out the recurrence a few times, until a pattern emerges. For instance, let's start with the familiar $T(n) = 2T(n/2) + O(n)$. Think of $O(n)$ as being $\leq cn$ for some constant c , so: $T(n) \leq 2T(n/2) + cn$. By repeatedly applying this rule, we can bound $T(n)$ in terms of $T(n/2)$, then $T(n/4)$, then $T(n/8)$, and so on, at each step getting closer to the value of $T(\cdot)$ we do know, namely $T(1) = O(1)$.

$$\begin{aligned}
 T(n) &\leq 2T(n/2) + cn \\
 &\leq 2[2T(n/4) + cn/2] + cn = 4T(n/4) + 2cn \\
 &\leq 4[2T(n/8) + cn/4] + 2cn = 8T(n/8) + 3cn \\
 &\leq 8[2T(n/16) + cn/8] + 3cn = 16T(n/16) + 4cn \\
 &\vdots
 \end{aligned}$$

A pattern is emerging... the general term is

$$T(n) \leq 2^k T(n/2^k) + kcn.$$

Plugging in $k = \log_2 n$, we get $T(n) \leq nT(1) + cn \log_2 n = O(n \log n)$.

- (a) Do the same thing for the recurrence $T(n) = 3T(n/2) + O(n)$. What is the general k th term in this case? And what value of k should be plugged in to get the answer?
 - (b) Now try the recurrence $T(n) = T(n-1) + O(1)$, a case which is not covered by the master theorem. Can you solve this too?
- 2.4. Suppose you are choosing between the following three algorithms:
- Algorithm *A* solves problems by dividing them into five subproblems of half the size, recursively solving each subproblem, and then combining the solutions in linear time.
 - Algorithm *B* solves problems of size n by recursively solving two subproblems of size $n-1$ and then combining the solutions in constant time.
 - Algorithm *C* solves problems of size n by dividing them into nine subproblems of size $n/3$, recursively solving each subproblem, and then combining the solutions in $O(n^2)$ time.

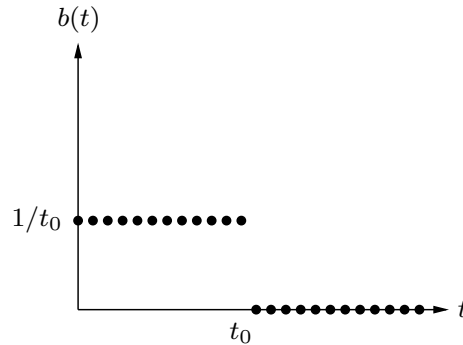
What are the running times of each of these algorithms (in big- O notation), and which would you choose?

- 2.5. Solve the following recurrence relations and give a Θ bound for each of them.

- (a) $T(n) = 2T(n/3) + 1$
- (b) $T(n) = 5T(n/4) + n$
- (c) $T(n) = 7T(n/7) + n$
- (d) $T(n) = 9T(n/3) + n^2$

- (e) $T(n) = 8T(n/2) + n^3$
- (f) $T(n) = 49T(n/25) + n^{3/2} \log n$
- (g) $T(n) = T(n-1) + 2$
- (h) $T(n) = T(n-1) + n^c$, where $c \geq 1$ is a constant
- (i) $T(n) = T(n-1) + c^n$, where $c > 1$ is some constant
- (j) $T(n) = 2T(n-1) + 1$
- (k) $T(n) = T(\sqrt{n}) + 1$

2.6. A linear, time-invariant system has the following impulse response:



- (a) Describe in words the effect of this system.
 - (b) What is the corresponding polynomial?
- 2.7. What is the sum of the n th roots of unity? What is their product if n is odd? If n is even?
- 2.8. Practice with the fast Fourier transform.
- (a) What is the FFT of $(1, 0, 0, 0)$? What is the appropriate value of ω in this case? And of which sequence is $(1, 0, 0, 0)$ the FFT?
 - (b) Repeat for $(1, 0, 1, -1)$.
- 2.9. Practice with polynomial multiplication by FFT.
- (a) Suppose that you want to multiply the two polynomials $x + 1$ and $x^2 + 1$ using the FFT. Choose an appropriate power of two, find the FFT of the two sequences, multiply the results componentwise, and compute the inverse FFT to get the final result.
 - (b) Repeat for the pair of polynomials $1 + x + 2x^2$ and $2 + 3x$.
- 2.10. Find the unique polynomial of degree 4 that takes on values $p(1) = 2, p(2) = 1, p(3) = 0, p(4) = 4$, and $p(5) = 0$. Write your answer in the coefficient representation.
- 2.11. In justifying our matrix multiplication algorithm (Section 2.5), we claimed the following block-wise property: if X and Y are $n \times n$ matrices, and

$$X = \begin{bmatrix} A & B \\ C & D \end{bmatrix}, \quad Y = \begin{bmatrix} E & F \\ G & H \end{bmatrix}.$$

where A, B, C, D, E, F, G , and H are $n/2 \times n/2$ submatrices, then the product XY can be expressed in terms of these blocks:

$$XY = \begin{bmatrix} A & B \\ C & D \end{bmatrix} \begin{bmatrix} E & F \\ G & H \end{bmatrix} = \begin{bmatrix} AE + BG & AF + BH \\ CE + DG & CF + DH \end{bmatrix}$$

Prove this property.

- 2.12. How many lines, as a function of n (in $\Theta(\cdot)$ form), does the following program print? Write a recurrence and solve it. You may assume n is a power of 2.

```
function f(n)
  if n > 1:
    print_line('still going')
    f(n/2)
    f(n/2)
```

- 2.13. A binary tree is *full* if all of its vertices have either zero or two children. Let B_n denote the number of full binary trees with n vertices.

- By drawing out all full binary trees with 3, 5, or 7 vertices, determine the exact values of B_3 , B_5 , and B_7 . Why have we left out even numbers of vertices, like B_4 ?
- For general n , derive a recurrence relation for B_n .
- Show by induction that B_n is $\Omega(2^n)$.

- 2.14. You are given an array of n elements, and you notice that some of the elements are duplicates; that is, they appear more than once in the array. Show how to remove all duplicates from the array in time $O(n \log n)$.

- 2.15. In our median-finding algorithm (Section 2.4), a basic primitive is the `split` operation, which takes as input an array S and a value v and then divides S into three sets: the elements less than v , the elements equal to v , and the elements greater than v . Show how to implement this `split` operation *in place*, that is, without allocating new memory.

- 2.16. You are given an infinite array $A[\cdot]$ in which the first n cells contain integers in sorted order and the rest of the cells are filled with ∞ . You are *not* given the value of n . Describe an algorithm that takes an integer x as input and finds a position in the array containing x , if such a position exists, in $O(\log n)$ time. (If you are disturbed by the fact that the array A has infinite length, assume instead that it is of length n , but that you don't know this length, and that the implementation of the array data type in your programming language returns the error message ∞ whenever elements $A[i]$ with $i > n$ are accessed.)

- 2.17. Given a sorted array of distinct integers $A[1, \dots, n]$, you want to find out whether there is an index i for which $A[i] = i$. Give a divide-and-conquer algorithm that runs in time $O(\log n)$.

- 2.18. Consider the task of searching a sorted array $A[1 \dots n]$ for a given element x : a task we usually perform by binary search in time $O(\log n)$. Show that any algorithm that accesses the array only via comparisons (that is, by asking questions of the form “is $A[i] \leq z$?”), must take $\Omega(\log n)$ steps.

- 2.19. A *k-way merge operation*. Suppose you have k sorted arrays, each with n elements, and you want to combine them into a single sorted array of kn elements.

- Here's one strategy: Using the merge procedure from Section 2.3, merge the first two arrays, then merge in the third, then merge in the fourth, and so on. What is the time complexity of this algorithm, in terms of k and n ?
- Give a more efficient solution to this problem, using divide-and-conquer.

- 2.20. Show that any array of integers $x[1 \dots n]$ can be sorted in $O(n + M)$ time, where

$$M = \max_i x_i - \min_i x_i.$$

For small M , this is linear time: why doesn't the $\Omega(n \log n)$ lower bound apply in this case?

2.21. *Mean and median.* One of the most basic tasks in statistics is to summarize a set of observations $\{x_1, x_2, \dots, x_n\} \subseteq \mathbb{R}$ by a single number. Two popular choices for this summary statistic are:

- The median, which we'll call μ_1
- The mean, which we'll call μ_2

(a) Show that the median is the value of μ that minimizes the function

$$\sum_i |x_i - \mu|.$$

You can assume for simplicity that n is odd. (*Hint:* Show that for any $\mu \neq \mu_1$, the function decreases if you move μ either slightly to the left or slightly to the right.)

(b) Show that the mean is the value of μ that minimizes the function

$$\sum_i (x_i - \mu)^2.$$

One way to do this is by calculus. Another method is to prove that for any $\mu \in \mathbb{R}$,

$$\sum_i (x_i - \mu)^2 = \sum_i (x_i - \mu_2)^2 + n(\mu - \mu_2)^2.$$

Notice how the function for μ_2 penalizes points that are far from μ much more heavily than the function for μ_1 . Thus μ_2 tries much harder to be close to *all* the observations. This might sound like a good thing at some level, but it is statistically undesirable because just a few outliers can severely throw off the estimate of μ_2 . It is therefore sometimes said that μ_1 is a more robust estimator than μ_2 . Worse than either of them, however, is μ_∞ , the value of μ that minimizes the function

$$\max_i |x_i - \mu|.$$

(c) Show that μ_∞ can be computed in $O(n)$ time (assuming the numbers x_i are small enough that basic arithmetic operations on them take unit time).

2.22. You are given two sorted lists of size m and n . Give an $O(\log m + \log n)$ time algorithm for computing the k th smallest element in the union of the two lists.

2.23. An array $A[1 \dots n]$ is said to have a *majority element* if more than half of its entries are the same. Given an array, the task is to design an efficient algorithm to tell whether the array has a majority element, and, if so, to find that element. The elements of the array are not necessarily from some ordered domain like the integers, and so there can be no comparisons of the form “is $A[i] > A[j]$?”. (Think of the array elements as GIF files, say.) However you *can* answer questions of the form: “is $A[i] = A[j]$?” in constant time.

(a) Show how to solve this problem in $O(n \log n)$ time. (*Hint:* Split the array A into two arrays A_1 and A_2 of half the size. Does knowing the majority elements of A_1 and A_2 help you figure out the majority element of A ? If so, you can use a divide-and-conquer approach.)

(b) Can you give a linear-time algorithm? (*Hint:* Here's another divide-and-conquer approach:

- Pair up the elements of A arbitrarily, to get $n/2$ pairs
- Look at each pair: if the two elements are different, discard both of them; if they are the same, keep just one of them

Show that after this procedure there are at most $n/2$ elements left, and that they have a majority element if and only if A does.)

2.24. On page 62 there is a high-level description of the quicksort algorithm.

- (a) Write down the pseudocode for quicksort.
- (b) Show that its *worst-case* running time on an array of size n is $\Theta(n^2)$.
- (c) Show that its *expected* running time satisfies the recurrence relation

$$T(n) \leq O(n) + \frac{1}{n} \sum_{i=1}^{n-1} (T(i) + T(n-i)).$$

Then, show that the solution to this recurrence is $O(n \log n)$.

2.25. In Section 2.1 we described an algorithm that multiplies two n -bit binary integers x and y in time n^a , where $a = \log_2 3$. Call this procedure `fastmultiply( $x, y$ )`.

- (a) We want to convert the decimal integer 10^n (a 1 followed by n zeros) into binary. Here is the algorithm (assume n is a power of 2):

```
function pwr2bin( $n$ )
  if  $n = 1$ : return 10102
  else:
     $z = ???$ 
    return fastmultiply( $z, z$ )
```

Fill in the missing details. Then give a recurrence relation for the running time of the algorithm, and solve the recurrence.

- (b) Next, we want to convert any decimal integer x with n digits (where n is a power of 2) into binary. The algorithm is the following:

```
function dec2bin( $x$ )
  if  $n = 1$ : return binary[ $x$ ]
  else:
    split  $x$  into two decimal numbers  $x_L, x_R$  with  $n/2$  digits each
    return ???
```

Here `binary[ $\cdot$ ]` is a vector that contains the binary representation of all one-digit integers. That is, `binary[0] = 02`, `binary[1] = 12`, up to `binary[9] = 10012`. Assume that a lookup in `binary` takes $O(1)$ time.

Fill in the missing details. Once again, give a recurrence for the running time of the algorithm, and solve it.

2.26. Professor F. Lake tells his class that it is asymptotically faster to square an n -bit integer than to multiply two n -bit integers. Should they believe him?

2.27. The *square* of a matrix A is its product with itself, AA .

- (a) Show that five multiplications are sufficient to compute the square of a 2×2 matrix.
- (b) What is wrong with the following algorithm for computing the square of an $n \times n$ matrix?
 “Use a divide-and-conquer approach as in Strassen’s algorithm, except that instead of getting 7 subproblems of size $n/2$, we now get 5 subproblems of size $n/2$ thanks to part (a). Using the same analysis as in Strassen’s algorithm, we can conclude that the algorithm runs in time $O(n^{\log_2 5})$.”

(c) In fact, squaring matrices is no easier than matrix multiplication. In this part, you will show that if $n \times n$ matrices can be squared in time $S(n) = O(n^c)$, then any two $n \times n$ matrices can be multiplied in time $O(n^c)$.

- i. Given two $n \times n$ matrices A and B , show that the matrix $AB + BA$ can be computed in time $3S(n) + O(n^2)$.
- ii. Given two $n \times n$ matrices X and Y , define the $2n \times 2n$ matrices A and B as follows:

$$A = \begin{bmatrix} X & 0 \\ 0 & 0 \end{bmatrix} \text{ and } B = \begin{bmatrix} 0 & Y \\ 0 & 0 \end{bmatrix}.$$

What is $AB + BA$, in terms of X and Y ?

- iii. Using (i) and (ii), argue that the product XY can be computed in time $3S(2n) + O(n^2)$. Conclude that matrix multiplication takes time $O(n^c)$.

2.28. The *Hadamard matrices* $H_0, H_1, H_2, \dots$ are defined as follows:

- H_0 is the 1×1 matrix $[1]$
- For $k > 0$, H_k is the $2^k \times 2^k$ matrix

$$H_k = \left[\begin{array}{c|c} H_{k-1} & H_{k-1} \\ \hline H_{k-1} & -H_{k-1} \end{array} \right]$$

Show that if v is a column vector of length $n = 2^k$, then the matrix-vector product $H_k v$ can be calculated using $O(n \log n)$ operations. Assume that all the numbers involved are small enough that basic arithmetic operations like addition and multiplication take unit time.

2.29. Suppose we want to evaluate the polynomial $p(x) = a_0 + a_1x + a_2x^2 + \dots + a_nx^n$ at point x .

- (a) Show that the following simple routine, known as *Horner's rule*, does the job and leaves the answer in z .

```

z = a_n
for i = n - 1 downto 0:
    z = zx + a_i

```

- (b) How many additions and multiplications does this routine use, as a function of n ? Can you find a polynomial for which an alternative method is substantially better?

2.30. This problem illustrates how to do the Fourier Transform (FT) in modular arithmetic, for example, modulo 7.

- (a) There is a number ω such that all the powers $\omega, \omega^2, \dots, \omega^6$ are distinct (modulo 7). Find this ω , and show that $\omega + \omega^2 + \dots + \omega^6 = 0$. (Interestingly, for any prime modulus there is such a number.)
- (b) Using the matrix form of the FT, produce the transform of the sequence $(0, 1, 1, 1, 5, 2)$ modulo 7; that is, multiply this vector by the matrix $M_6(\omega)$, for the value of ω you found earlier. In the matrix multiplication, all calculations should be performed modulo 7.
- (c) Write down the matrix necessary to perform the inverse FT. Show that multiplying by this matrix returns the original sequence. (Again all arithmetic should be performed modulo 7.)
- (d) Now show how to multiply the polynomials $x^2 + x + 1$ and $x^3 + 2x - 1$ using the FT modulo 7.

2.31. In Section 1.2.3, we studied Euclid's algorithm for computing the *greatest common divisor* (gcd) of two positive integers: the largest integer which divides them both. Here we will look at an alternative algorithm based on divide-and-conquer.

(a) Show that the following rule is true.

$$\gcd(a, b) = \begin{cases} 2 \gcd(a/2, b/2) & \text{if } a, b \text{ are even} \\ \gcd(a, b/2) & \text{if } a \text{ is odd, } b \text{ is even} \\ \gcd((a-b)/2, b) & \text{if } a, b \text{ are odd} \end{cases}$$

(b) Give an efficient divide-and-conquer algorithm for greatest common divisor.

(c) How does the efficiency of your algorithm compare to Euclid's algorithm if a and b are n -bit integers? (In particular, since n might be large you cannot assume that basic arithmetic operations like addition take constant time.)

2.32. In this problem we will develop a divide-and-conquer algorithm for the following geometric task.

CLOSEST PAIR

Input: A set of points in the plane, $\{p_1 = (x_1, y_1), p_2 = (x_2, y_2), \dots, p_n = (x_n, y_n)\}$

Output: The closest pair of points: that is, the pair $p_i \neq p_j$ for which the distance between p_i and p_j , that is,

$$\sqrt{(x_i - x_j)^2 + (y_i - y_j)^2},$$

is minimized.

For simplicity, assume that n is a power of two, and that all the x -coordinates x_i are distinct, as are the y -coordinates.

Here's a high-level overview of the algorithm:

- Find a value x for which exactly half the points have $x_i < x$, and half have $x_i > x$. On this basis, split the points into two groups, L and R .
- Recursively find the closest pair in L and in R . Say these pairs are $p_L, q_L \in L$ and $p_R, q_R \in R$, with distances d_L and d_R respectively. Let d be the smaller of these two distances.
- It remains to be seen whether there is a point in L and a point in R that are less than distance d apart from each other. To this end, discard all points with $x_i < x - d$ or $x_i > x + d$ and sort the remaining points by y -coordinate.
- Now, go through this sorted list, and for each point, compute its distance to the *seven* subsequent points in the list. Let p_M, q_M be the closest pair found in this way.
- The answer is one of the three pairs $\{p_L, q_L\}, \{p_R, q_R\}, \{p_M, q_M\}$, whichever is closest.

- (a) In order to prove the correctness of this algorithm, start by showing the following property: any square of size $d \times d$ in the plane contains at most four points of L .
- (b) Now show that the algorithm is correct. The only case which needs careful consideration is when the closest pair is split between L and R .
- (c) Write down the pseudocode for the algorithm, and show that its running time is given by the recurrence:

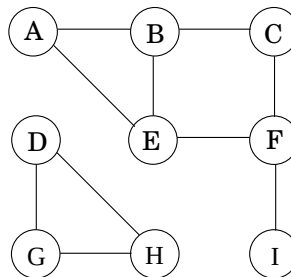
$$T(n) = 2T(n/2) + O(n \log n).$$

Show that the solution to this recurrence is $O(n \log^2 n)$.

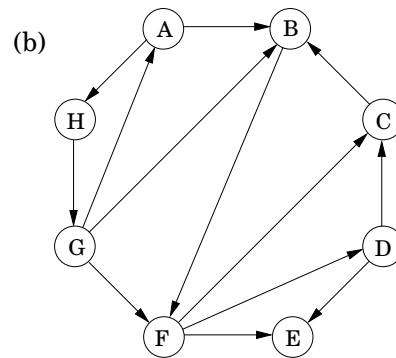
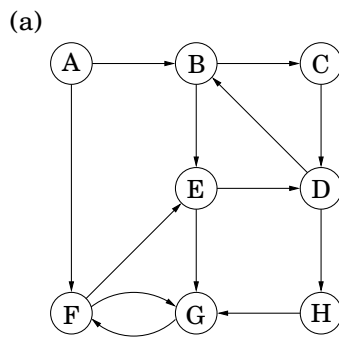
- (d) Can you bring the running time down to $O(n \log n)$?

Exercises

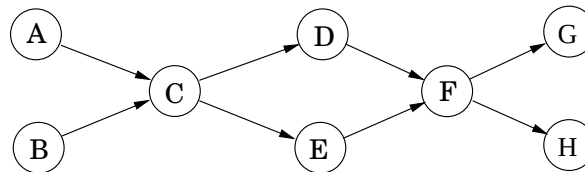
- 3.1. Perform a depth-first search on the following graph; whenever there's a choice of vertices, pick the one that is alphabetically first. Classify each edge as a tree edge or back edge, and give the pre and post number of each vertex.



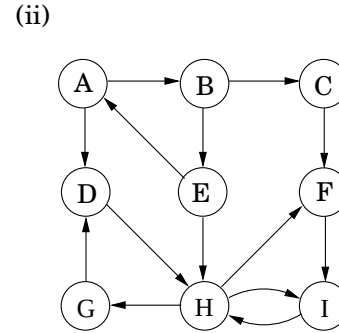
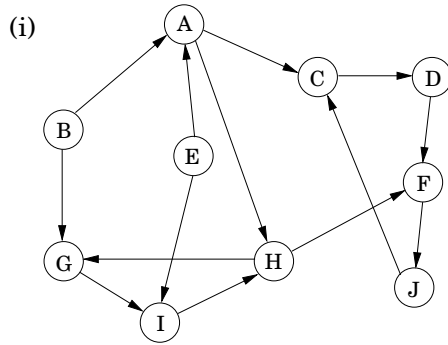
- 3.2. Perform depth-first search on each of the following graphs; whenever there's a choice of vertices, pick the one that is alphabetically first. Classify each edge as a tree edge, forward edge, back edge, or cross edge, and give the pre and post number of each vertex.



- 3.3. Run the DFS-based topological ordering algorithm on the following graph. Whenever you have a choice of vertices to explore, always pick the one that is alphabetically first.



- Indicate the pre and post numbers of the nodes.
 - What are the sources and sinks of the graph?
 - What topological ordering is found by the algorithm?
 - How many topological orderings does this graph have?
- 3.4. Run the strongly connected components algorithm on the following directed graphs G . When doing DFS on G^R : whenever there is a choice of vertices to explore, always pick the one that is alphabetically first.



In each case answer the following questions.

- (a) In what order are the strongly connected components (SCCs) found?
 - (b) Which are source SCCs and which are sink SCCs?
 - (c) Draw the “metagraph” (each meta-node is an SCC of G).
 - (d) What is the minimum number of edges you must add to this graph to make it strongly connected?
- 3.5. The *reverse* of a directed graph $G = (V, E)$ is another directed graph $G^R = (V, E^R)$ on the same vertex set, but with all edges reversed; that is, $E^R = \{(v, u) : (u, v) \in E\}$.
Give a linear-time algorithm for computing the reverse of a graph in adjacency list format.
- 3.6. In an undirected graph, the *degree* $d(u)$ of a vertex u is the number of neighbors u has, or equivalently, the number of edges incident upon it. In a directed graph, we distinguish between the *indegree* $d_{in}(u)$, which is the number of edges into u , and the *outdegree* $d_{out}(u)$, the number of edges leaving u .
- (a) Show that in an undirected graph, $\sum_{u \in V} d(u) = 2|E|$.
 - (b) Use part (a) to show that in an undirected graph, there must be an even number of vertices whose degree is odd.
 - (c) Does a similar statement hold for the number of vertices with odd indegree in a directed graph?
- 3.7. A *bipartite graph* is a graph $G = (V, E)$ whose vertices can be partitioned into two sets ($V = V_1 \cup V_2$ and $V_1 \cap V_2 = \emptyset$) such that there are no edges between vertices in the same set (for instance, if $u, v \in V_1$, then there is no edge between u and v).
- (a) Give a linear-time algorithm to determine whether an undirected graph is bipartite.
 - (b) There are many other ways to formulate this property. For instance, an undirected graph is bipartite if and only if it can be colored with just two colors.
Prove the following formulation: an undirected graph is bipartite if and only if it contains no cycles of odd length.
 - (c) At most how many colors are needed to color in an undirected graph with exactly *one* odd-length cycle?
- 3.8. *Pouring water.* We have three containers whose sizes are 10 pints, 7 pints, and 4 pints, respectively. The 7-pint and 4-pint containers start out full of water, but the 10-pint container is initially empty. We are allowed one type of operation: pouring the contents of one container into another, stopping only when the source container is empty or the destination container is full. We want to know if there is a sequence of pourings that leaves exactly 2 pints in the 7- or 4-pint container.

- (a) Model this as a graph problem: give a precise definition of the graph involved and state the specific question about this graph that needs to be answered.
 - (b) What algorithm should be applied to solve the problem?
 - (c) Find the answer by applying the algorithm.
- 3.9. For each node u in an undirected graph, let $\text{twodegree}[u]$ be the sum of the degrees of u 's neighbors. Show how to compute the entire array of $\text{twodegree}[\cdot]$ values in linear time, given a graph in adjacency list format.
- 3.10. Rewrite the `explore` procedure (Figure 3.3) so that it is non-recursive (that is, explicitly use a stack). The calls to `previsit` and `postvisit` should be positioned so that they have the same effect as in the recursive procedure.
- 3.11. Design a linear-time algorithm which, given an undirected graph G and a particular edge e in it, determines whether G has a cycle containing e .
- 3.12. Either prove or give a counterexample: if $\{u, v\}$ is an edge in an undirected graph, and during depth-first search $\text{post}(u) < \text{post}(v)$, then v is an ancestor of u in the DFS tree.
- 3.13. *Undirected vs. directed connectivity.*
- (a) Prove that in any connected undirected graph $G = (V, E)$ there is a vertex $v \in V$ whose removal leaves G connected. (*Hint:* Consider the DFS search tree for G .)
 - (b) Give an example of a strongly connected directed graph $G = (V, E)$ such that, for every $v \in V$, removing v from G leaves a directed graph that is not strongly connected.
 - (c) In an undirected graph with 2 connected components it is always possible to make the graph connected by adding only one edge. Give an example of a directed graph with two strongly connected components such that no addition of one edge can make the graph strongly connected.
- 3.14. The chapter suggests an alternative algorithm for linearization (topological sorting), which repeatedly removes source nodes from the graph (page 97). Show that this algorithm can be implemented in linear time.
- 3.15. The police department in the city of Computopia has made all streets one-way. The mayor contends that there is still a way to drive legally from any intersection in the city to any other intersection, but the opposition is not convinced. A computer program is needed to determine whether the mayor is right. However, the city elections are coming up soon, and there is just enough time to run a *linear-time* algorithm.
- (a) Formulate this problem graph-theoretically, and explain why it can indeed be solved in linear time.
 - (b) Suppose it now turns out that the mayor's original claim is false. She next claims something weaker: if you start driving from town hall, navigating one-way streets, then no matter where you reach, there is always a way to drive legally back to the town hall. Formulate this weaker property as a graph-theoretic problem, and carefully show how it too can be checked in linear time.
- 3.16. Suppose a CS curriculum consists of n courses, all of them mandatory. The prerequisite graph G has a node for each course, and an edge from course v to course w if and only if v is a prerequisite for w . Find an algorithm that works directly with this graph representation, and computes the minimum number of semesters necessary to complete the curriculum (assume that a student can take any number of courses in one semester). The running time of your algorithm should be linear.

3.17. *Infinite paths.* Let $G = (V, E)$ be a directed graph with a designated “start vertex” $s \in V$, a set $V_G \subseteq V$ of “good” vertices, and a set $V_B \subseteq V$ of “bad” vertices. An *infinite trace* p of G is an infinite sequence $v_0 v_1 v_2 \dots$ of vertices $v_i \in V$ such that (1) $v_0 = s$, and (2) for all $i \geq 0$, $(v_i, v_{i+1}) \in E$. That is, p is an infinite path in G starting at vertex s . Since the set V of vertices is finite, every infinite trace of G must visit some vertices infinitely often.

- (a) If p is an infinite trace, let $\text{Inf}(p) \subseteq V$ be the set of vertices that occur infinitely often in p . Show that $\text{Inf}(p)$ is a subset of a strongly connected component of G .
- (b) Describe an algorithm that determines if G has an infinite trace.
- (c) Describe an algorithm that determines if G has an infinite trace that visits some good vertex in V_G infinitely often.
- (d) Describe an algorithm that determines if G has an infinite trace that visits some good vertex in V_G infinitely often, but visits no bad vertex in V_B infinitely often.

3.18. You are given a binary tree $T = (V, E)$ (in adjacency list format), along with a designated root node $r \in V$. Recall that u is said to be an *ancestor* of v in the rooted tree, if the path from r to v in T passes through u .

You wish to preprocess the tree so that queries of the form “is u an ancestor of v ?” can be answered in constant time. The preprocessing itself should take linear time. How can this be done?

3.19. As in the previous problem, you are given a binary tree $T = (V, E)$ with designated root node. In addition, there is an array $x[\cdot]$ with a value for each node in V . Define a new array $z[\cdot]$ as follows: for each $u \in V$,

$$z[u] = \text{the maximum of the } x\text{-values associated with } u\text{'s descendants.}$$

Give a linear-time algorithm which calculates the entire z -array.

3.20. You are given a tree $T = (V, E)$ along with a designated root node $r \in V$. The *parent* of any node $v \neq r$, denoted $p(v)$, is defined to be the node adjacent to v in the path from r to v . By convention, $p(r) = r$. For $k > 1$, define $p^k(v) = p^{k-1}(p(v))$ and $p^1(v) = p(v)$ (so $p^k(v)$ is the k th ancestor of v).

Each vertex v of the tree has an associated non-negative integer label $l(v)$. Give a linear-time algorithm to update the labels of all the vertices in T according to the following rule: $l_{\text{new}}(v) = l(p^{l(v)}(v))$.

3.21. Give a linear-time algorithm to find an odd-length cycle in a *directed* graph. (*Hint:* First solve this problem under the assumption that the graph is strongly connected.)

3.22. Give an efficient algorithm which takes as input a directed graph $G = (V, E)$, and determines whether or not there is a vertex $s \in V$ from which all other vertices are reachable.

3.23. Give an efficient algorithm that takes as input a directed acyclic graph $G = (V, E)$, and two vertices $s, t \in V$, and outputs the number of different directed paths from s to t in G .

3.24. Give a linear-time algorithm for the following task.

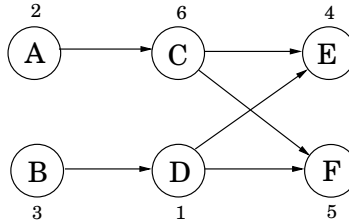
Input: A directed acyclic graph G

Question: Does G contain a directed path that touches every vertex exactly once?

3.25. You are given a directed graph in which each node $u \in V$ has an associated *price* p_u which is a positive integer. Define the array *cost* as follows: for each $u \in V$,

$$\text{cost}[u] = \text{price of the cheapest node reachable from } u \text{ (including } u \text{ itself).}$$

For instance, in the graph below (with prices shown for each vertex), the cost values of the nodes A, B, C, D, E, F are 2, 1, 4, 1, 4, 5, respectively.

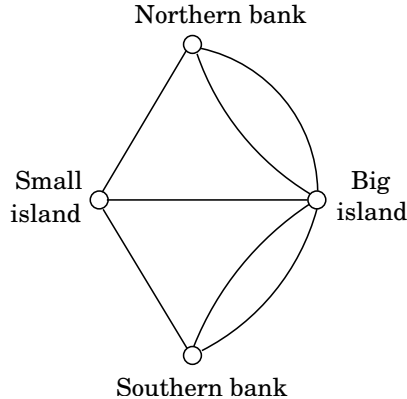


Your goal is to design an algorithm that fills in the *entire* cost array (i.e., for all vertices).

- (a) Give a linear-time algorithm that works for directed *acyclic* graphs. (*Hint*: Handle the vertices in a particular *order*.)
 - (b) Extend this to a linear-time algorithm that works for all directed graphs. (*Hint*: Recall the “two-tiered” structure of directed graphs.)
- 3.26. An *Eulerian tour* in an undirected graph is a cycle that is allowed to pass through each vertex multiple times, but must use each edge exactly once.

This simple concept was used by Euler in 1736 to solve the famous Königsberg bridge problem, which launched the field of graph theory. The city of Königsberg (now called Kaliningrad, in western Russia) is the meeting point of two rivers with a small island in the middle. There are seven bridges across the rivers, and a popular recreational question of the time was to determine whether it is possible to perform a tour in which each bridge is crossed *exactly once*.

Euler formulated the relevant information as a graph with four nodes (denoting land masses) and seven edges (denoting bridges), as shown here.



Notice an unusual feature of this problem: multiple edges between certain pairs of nodes.

- (a) Show that an undirected graph has an Eulerian tour if and only if all its vertices have even degree. Conclude that there is no Eulerian tour of the Königsberg bridges.
 - (b) An *Eulerian path* is a path which uses each edge exactly once. Can you give a similar if-and-only-if characterization of which undirected graphs have Eulerian paths?
 - (c) Can you give an analog of part (a) for *directed* graphs?
- 3.27. Two paths in a graph are called *edge-disjoint* if they have no edges in common. Show that in any undirected graph, it is possible to pair up the vertices of odd degree and find paths between each such pair so that all these paths are edge-disjoint.

- 3.28. In the 2SAT problem, you are given a set of *clauses*, where each clause is the disjunction (OR) of two literals (a literal is a Boolean variable or the negation of a Boolean variable). You are looking for a way to assign a value `true` or `false` to each of the variables so that *all* clauses are satisfied – that is, there is at least one true literal in each clause. For example, here’s an instance of 2SAT:

$$(x_1 \vee \bar{x}_2) \wedge (\bar{x}_1 \vee \bar{x}_3) \wedge (x_1 \vee x_2) \wedge (\bar{x}_3 \vee x_4) \wedge (\bar{x}_1 \vee x_4).$$

This instance has a satisfying assignment: set x_1 , x_2 , x_3 , and x_4 to `true`, `false`, `false`, and `true`, respectively.

- (a) Are there other satisfying truth assignments of this 2SAT formula? If so, find them all.
- (b) Give an instance of 2SAT with four variables, and with no satisfying assignment.

The purpose of this problem is to lead you to a way of solving 2SAT efficiently by reducing it to the problem of finding the strongly connected components of a directed graph. Given an instance I of 2SAT with n variables and m clauses, construct a directed graph $G_I = (V, E)$ as follows.

- G_I has $2n$ nodes, one for each variable and its negation.
- G_I has $2m$ edges: for each clause $(\alpha \vee \beta)$ of I (where α, β are literals), G_I has an edge from the negation of α to β , and one from the negation of β to α .

Note that the clause $(\alpha \vee \beta)$ is equivalent to either of the implications $\bar{\alpha} \Rightarrow \beta$ or $\bar{\beta} \Rightarrow \alpha$. In this sense, G_I records all implications in I .

- (c) Carry out this construction for the instance of 2SAT given above, and for the instance you constructed in (b).
 - (d) Show that if G_I has a strongly connected component containing both x and $\bar{x}$ for some variable x , then I has no satisfying assignment.
 - (e) Now show the converse of (d): namely, that if none of G_I ’s strongly connected components contain both a literal and its negation, then the instance I must be satisfiable. (*Hint*: Assign values to the variables as follows: repeatedly pick a sink strongly connected component of G_I . Assign value `true` to all literals in the sink, assign `false` to their negations, and delete all of these. Show that this ends up discovering a satisfying assignment.)
 - (f) Conclude that there is a linear-time algorithm for solving 2SAT.
- 3.29. Let S be a finite set. A binary *relation* on S is simply a collection R of ordered pairs $(x, y) \in S \times S$. For instance, S might be a set of people, and each such pair $(x, y) \in R$ might mean “ x knows y .”
- An *equivalence relation* is a binary relation which satisfies three properties:

- Reflexivity: $(x, x) \in R$ for all $x \in S$
- Symmetry: if $(x, y) \in R$ then $(y, x) \in R$
- Transitivity: if $(x, y) \in R$ and $(y, z) \in R$ then $(x, z) \in R$

For instance, the binary relation “has the same birthday as” is an equivalence relation, whereas “is the father of” is not, since it violates all three properties.

Show that an equivalence relation partitions set S into disjoint groups $S_1, S_2, \dots, S_k$ (in other words, $S = S_1 \cup S_2 \cup \dots \cup S_k$ and $S_i \cap S_j = \emptyset$ for all $i \neq j$) such that:

- Any two members of a group are related, that is, $(x, y) \in R$ for any $x, y \in S_i$, for any i .
- Members of different groups are not related, that is, for all $i \neq j$, for all $x \in S_i$ and $y \in S_j$, we have $(x, y) \notin R$.

(*Hint*: Represent an equivalence relation by an undirected graph.)

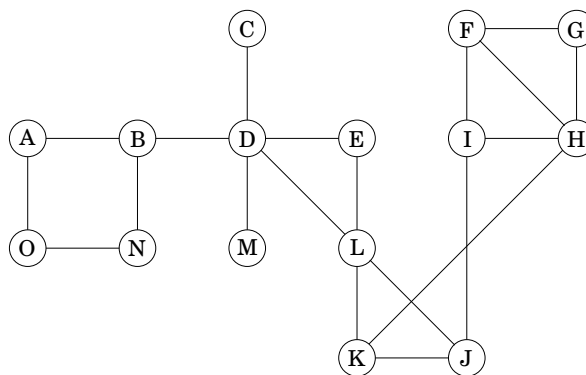
- 3.30. On page 97, we defined the binary relation “connected” on the set of vertices of a *directed* graph. Show that this is an equivalence relation (see Exercise 3.29), and conclude that it partitions the vertices into disjoint strongly connected components.
- 3.31. *Biconnected components* Let $G = (V, E)$ be an undirected graph. For any two edges $e, e' \in E$, we'll say $e \sim e'$ if either $e = e'$ or there is a (simple) cycle containing both e and e' .

(a) Show that $\sim$ is an equivalence relation (recall Exercise 3.29) on the edges.

The equivalence classes into which this relation partitions the edges are called the *biconnected components* of G . A *bridge* is an edge which is in a biconnected component all by itself.

A *separating vertex* is a vertex whose removal disconnects the graph.

- (b) Partition the edges of the graph below into biconnected components, and identify the bridges and separating vertices.



Not only do biconnected components partition the edges of the graph, they also *almost* partition the vertices in the following sense.

- (c) Associate with each biconnected component all the vertices that are endpoints of its edges. Show that the vertices corresponding to two different biconnected components are either disjoint or intersect in a single separating vertex.
- (d) Collapse each biconnected component into a single meta-node, and retain individual nodes for each separating vertex. (So there are edges between each component-node and its separating vertices.) Show that the resulting graph is a tree.

DFS can be used to identify the biconnected components, bridges, and separating vertices of a graph in linear time.

- (e) Show that the root of the DFS tree is a separating vertex if and only if it has more than one child in the tree.
- (f) Show that a non-root vertex v of the DFS tree is a separating vertex if and only if it has a child v' none of whose descendants (including itself) has a backedge to a proper ancestor of v .
- (g) For each vertex u define:

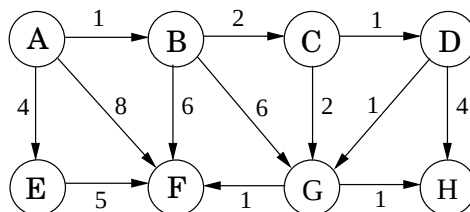
$$\text{low}(u) = \min \begin{cases} \text{pre}(u) \\ \text{pre}(w) \end{cases} \quad \text{where } (v, w) \text{ is a backedge for some descendant } v \text{ of } u$$

Show that the entire array of low values can be computed in linear time.

- (h) Show how to compute all separating vertices, bridges, and biconnected components of a graph in linear time. (*Hint:* Use low to identify separating vertices, and run another DFS with an extra stack of edges to remove biconnected components one at a time.)

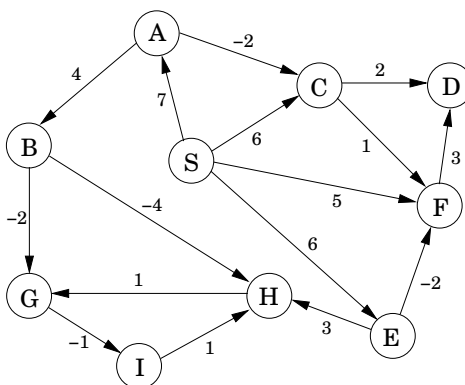
Exercises

4.1. Suppose Dijkstra's algorithm is run on the following graph, starting at node A.



- Draw a table showing the intermediate distance values of all the nodes at each iteration of the algorithm.
- Show the final shortest-path tree.

4.2. Just like the previous problem, but this time with the Bellman-Ford algorithm.



4.3. *Squares.* Design and analyze an algorithm that takes as input an undirected graph $G = (V, E)$ and determines whether G contains a simple cycle (that is, a cycle which doesn't intersect itself) of length four. Its running time should be at most $O(|V|^3)$.

You may assume that the input graph is represented either as an adjacency matrix or with adjacency lists, whichever makes your algorithm simpler.

4.4. Here's a proposal for how to find the length of the shortest cycle in an undirected graph with unit edge lengths.

When a back edge, say (v, w) , is encountered during a depth-first search, it forms a cycle with the tree edges from w to v . The length of the cycle is $\text{level}[v] - \text{level}[w] + 1$, where the level of a vertex is its distance in the DFS tree from the root vertex. This suggests the following algorithm:

- Do a depth-first search, keeping track of the level of each vertex.
- Each time a back edge is encountered, compute the cycle length and save it if it is smaller than the shortest one previously seen.

Show that this strategy does not always work by providing a counterexample as well as a brief (one or two sentence) explanation.

4.5. Often there are multiple shortest paths between two nodes of a graph. Give a linear-time algorithm for the following task.

Input: Undirected graph $G = (V, E)$ with unit edge lengths; nodes $u, v \in V$.

Output: The number of distinct shortest paths from u to v .

- 4.6. Prove that for the array `prev` computed by Dijkstra's algorithm, the edges $\{u, \text{prev}[u]\}$ (for all $u \in V$) form a tree.
- 4.7. You are given a directed graph $G = (V, E)$ with (possibly negative) weighted edges, along with a specific node $s \in V$ and a tree $T = (V, E'), E' \subseteq E$. Give an algorithm that checks whether T is a shortest-path tree for G with starting point s . Your algorithm should run in linear time.
- 4.8. Professor F. Lake suggests the following algorithm for finding the shortest path from node s to node t in a directed graph with some negative edges: add a large constant to each edge weight so that all the weights become positive, then run Dijkstra's algorithm starting at node s , and return the shortest path found to node t .
- Is this a valid method? Either prove that it works correctly, or give a counterexample.
- 4.9. Consider a directed graph in which the only negative edges are those that leave s ; all other edges are positive. Can Dijkstra's algorithm, started at s , fail on such a graph? Prove your answer.
- 4.10. You are given a directed graph with (possibly negative) weighted edges, in which the shortest path between any two vertices is guaranteed to have at most k edges. Give an algorithm that finds the shortest path between two vertices u and v in $O(k|E|)$ time.
- 4.11. Give an algorithm that takes as input a directed graph with positive edge lengths, and returns the length of the shortest cycle in the graph (if the graph is acyclic, it should say so). Your algorithm should take time at most $O(|V|^3)$.
- 4.12. Give an $O(|V|^2)$ algorithm for the following task.

Input: An undirected graph $G = (V, E)$; edge lengths $l_e > 0$; an edge $e \in E$.

Output: The length of the shortest cycle containing edge e .

- 4.13. You are given a set of cities, along with the pattern of highways between them, in the form of an undirected graph $G = (V, E)$. Each stretch of highway $e \in E$ connects two of the cities, and you know its length in miles, l_e . You want to get from city s to city t . There's one problem: your car can only hold enough gas to cover L miles. There are gas stations in each city, but not between cities. Therefore, you can only take a route if every one of its edges has length $l_e \leq L$.
- (a) Given the limitation on your car's fuel tank capacity, show how to determine in linear time whether there is a feasible route from s to t .
- (b) You are now planning to buy a new car, and you want to know the minimum fuel tank capacity that is needed to travel from s to t . Give an $O((|V| + |E|) \log |V|)$ algorithm to determine this.
- 4.14. You are given a strongly connected directed graph $G = (V, E)$ with positive edge weights along with a particular node $v_0 \in V$. Give an efficient algorithm for finding shortest paths between *all pairs of nodes*, with the one restriction that these paths must all pass through v_0 .
- 4.15. Shortest paths are not always unique: sometimes there are two or more different paths with the minimum possible length. Show how to solve the following problem in $O((|V| + |E|) \log |V|)$ time.

Input: An undirected graph $G = (V, E)$; edge lengths $l_e > 0$; starting vertex $s \in V$.

Output: A Boolean array `usp[·]`: for each node u , the entry `usp[u]` should be `true` if and only if there is a *unique* shortest path from s to u . (Note: `usp[s] = true`.)

Figure 4.16 Operations on a binary heap.

procedure insert(h, x)
bubbleup($h, x, |h| + 1$)

procedure decreasekey(h, x)
bubbleup($h, x, h^{-1}(x)$)

function deletemin(h)
if $|h| = 0$:
 return null
else:
 $x = h(1)$
 siftdown($h, h(|h|), 1$)
 return x

function makeheap(S)
 $h =$ empty array of size $|S|$
for $x \in S$:
 $h(|h| + 1) = x$
for $i = |S|$ downto 1:
 siftdown($h, h(i), i$)
return h

procedure bubbleup(h, x, i)
(place element x in position i of h , and let it bubble up)
 $p = \lceil i/2 \rceil$
while $i \neq 1$ and $\text{key}(h(p)) > \text{key}(x)$:
 $h(i) = h(p)$; $i = p$; $p = \lceil i/2 \rceil$
 $h(i) = x$

procedure siftdown(h, x, i)
(place element x in position i of h , and let it sift down)
 $c = \text{minchild}(h, i)$
while $c \neq 0$ and $\text{key}(h(c)) < \text{key}(x)$:
 $h(i) = h(c)$; $i = c$; $c = \text{minchild}(h, i)$
 $h(i) = x$

function minchild(h, i)
(return the index of the smallest child of $h(i)$)
if $2i > |h|$:
 return 0 (no children)
else:
 return $\arg \min\{\text{key}(h(j)) : 2i \leq j \leq \min\{|h|, 2i + 1\}\}$

4.16. Section 4.5.2 describes a way of storing a complete binary tree of n nodes in an array indexed by $1, 2, \dots, n$.

- (a) Consider the node at position j of the array. Show that its parent is at position $\lfloor j/2 \rfloor$ and its children are at $2j$ and $2j + 1$ (if these numbers are $\leq n$).
- (b) What the corresponding indices when a complete d -ary tree is stored in an array?

Figure 4.16 shows pseudocode for a binary heap, modeled on an exposition by R.E. Tarjan.² The heap is stored as an array h , which is assumed to support two constant-time operations:

- $|h|$, which returns the number of elements currently in the array;
- h^{-1} , which returns the position of an element within the array.

The latter can always be achieved by maintaining the values of h^{-1} as an auxiliary array.

- (c) Show that the makeheap procedure takes $O(n)$ time when called on a set of n elements. What is the worst-case input? (*Hint:* Start by showing that the running time is at most $\sum_{i=1}^n \log(n/i)$.)
- (a) What needs to be changed to adapt this pseudocode to d -ary heaps?

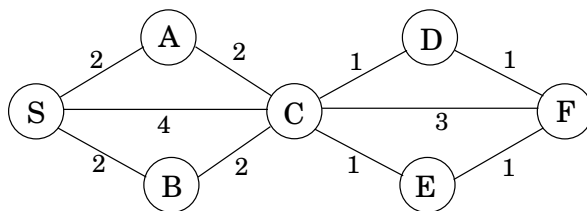
4.17. Suppose we want to run Dijkstra's algorithm on a graph whose edge weights are integers in the range $0, 1, \dots, W$, where W is a relatively small number.

- (a) Show how Dijkstra's algorithm can be made to run in time $O(W|V| + |E|)$.
- (b) Show an alternative implementation that takes time just $O((|V| + |E|) \log W)$.

4.18. In cases where there are several different shortest paths between two nodes (and edges have varying lengths), the most convenient of these paths is often *the one with fewest edges*. For instance, if nodes represent cities and edge lengths represent costs of flying between cities, there might be many ways to get from city s to city t which all have the same cost. The most convenient of these alternatives is the one which involves the fewest stopovers. Accordingly, for a specific starting node s , define

$\text{best}[u] = \text{minimum number of edges in a shortest path from } s \text{ to } u.$

In the example below, the best values for nodes S, A, B, C, D, E, F are 0, 1, 1, 1, 2, 2, 3, respectively.



Give an efficient algorithm for the following problem.

Input: Graph $G = (V, E)$; positive edge lengths l_e ; starting node $s \in V$.

Output: The values of $\text{best}[u]$ should be set for *all* nodes $u \in V$.

²See: R. E. Tarjan, *Data Structures and Network Algorithms*, Society for Industrial and Applied Mathematics, 1983.

- 4.19. *Generalized shortest-paths problem.* In Internet routing, there are delays on lines but also, more significantly, delays at routers. This motivates a generalized shortest-paths problem.

Suppose that in addition to having edge lengths $\{l_e : e \in E\}$, a graph also has *vertex costs* $\{c_v : v \in V\}$. Now define the cost of a path to be the sum of its edge lengths, *plus* the costs of all vertices on the path (including the endpoints). Give an efficient algorithm for the following problem.

Input: A directed graph $G = (V, E)$; positive edge lengths l_e and positive vertex costs c_v ; a starting vertex $s \in V$.

Output: An array $\text{cost}[\cdot]$ such that for every vertex u , $\text{cost}[u]$ is the least cost of any path from s to u (i.e., the cost of the cheapest path), under the definition above.

Notice that $\text{cost}[s] = c_s$.

- 4.20. There is a network of roads $G = (V, E)$ connecting a set of cities V . Each road in E has an associated length l_e . There is a proposal to add one new road to this network, and there is a list E' of pairs of cities between which the new road can be built. Each such potential road $e' \in E'$ has an associated length. As a designer for the public works department you are asked to determine the road $e' \in E'$ whose addition to the existing network G would result in the maximum decrease in the driving distance between two fixed cities s and t in the network. Give an efficient algorithm for solving this problem.
- 4.21. Shortest path algorithms can be applied in currency trading. Let $c_1, c_2, \dots, c_n$ be various currencies; for instance, c_1 might be dollars, c_2 pounds, and c_3 lire. For any two currencies c_i and c_j , there is an exchange rate $r_{i,j}$; this means that you can purchase $r_{i,j}$ units of currency c_j in exchange for one unit of c_i . These exchange rates satisfy the condition that $r_{i,j} \cdot r_{j,i} < 1$, so that if you start with a unit of currency c_i , change it into currency c_j and then convert back to currency c_i , you end up with less than one unit of currency c_i (the difference is the cost of the transaction).

- (a) Give an efficient algorithm for the following problem: Given a set of exchange rates $r_{i,j}$, and two currencies s and t , find the most advantageous sequence of currency exchanges for converting currency s into currency t . Toward this goal, you should represent the currencies and rates by a graph whose edge lengths are real numbers.

The exchange rates are updated frequently, reflecting the demand and supply of the various currencies. Occasionally the exchange rates satisfy the following property: there is a sequence of currencies $c_{i_1}, c_{i_2}, \dots, c_{i_k}$ such that $r_{i_1, i_2} \cdot r_{i_2, i_3} \cdots r_{i_{k-1}, i_k} \cdot r_{i_k, i_1} > 1$. This means that by starting with a unit of currency c_{i_1} and then successively converting it to currencies $c_{i_2}, c_{i_3}, \dots, c_{i_k}$, and finally back to c_{i_1} , you would end up with more than one unit of currency c_{i_1} . Such anomalies last only a fraction of a minute on the currency exchange, but they provide an opportunity for risk-free profits.

- (b) Give an efficient algorithm for detecting the presence of such an anomaly. Use the graph representation you found above.

- 4.22. *The tramp steamer problem.* You are the owner of a steamship that can ply between a group of port cities V . You make money at each port: a visit to city i earns you a profit of p_i dollars. Meanwhile, the transportation cost from port i to port j is $c_{ij} > 0$. You want to find a cyclic route in which the ratio of profit to cost is maximized.

To this end, consider a directed graph $G = (V, E)$ whose nodes are ports, and which has edges between each pair of ports. For any cycle C in this graph, the profit-to-cost ratio is

$$r(C) = \frac{\sum_{(i,j) \in C} p_j}{\sum_{(i,j) \in C} c_{ij}}.$$

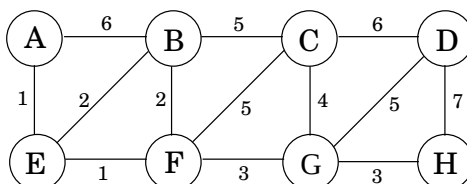
Let r^* be the maximum ratio achievable by a simple cycle. One way to determine r^* is by binary search: by first guessing some ratio r , and then testing whether it is too large or too small.

Consider any positive $r > 0$. Give each edge (i, j) a weight of $w_{ij} = rc_{ij} - p_j$.

- (a) Show that if there is a cycle of negative weight, then $r < r^*$.
- (b) Show that if all cycles in the graph have strictly positive weight, then $r > r^*$.
- (c) Give an efficient algorithm that takes as input a desired accuracy $\epsilon > 0$ and returns a simple cycle C for which $r(C) \geq r^* - \epsilon$. Justify the correctness of your algorithm and analyze its running time in terms of $|V|$, ϵ , and $R = \max_{(i,j) \in E} (p_j / c_{ij})$.

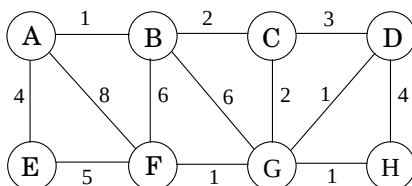
Exercises

5.1. Consider the following graph.



- What is the cost of its minimum spanning tree?
- How many minimum spanning trees does it have?
- Suppose Kruskal's algorithm is run on this graph. In what order are the edges added to the MST? For each edge in this sequence, give a cut that justifies its addition.

5.2. Suppose we want to find the minimum spanning tree of the following graph.



- Run Prim's algorithm; whenever there is a choice of nodes, always use alphabetic ordering (e.g., start from node A). Draw a table showing the intermediate values of the cost array.
- Run Kruskal's algorithm on the same graph. Show how the disjoint-sets data structure looks at every intermediate stage (including the structure of the directed trees), assuming path compression is used.

5.3. Design a linear-time algorithm for the following task.

Input: A connected, undirected graph G .

Question: Is there an edge you can remove from G while still leaving G connected?

Can you reduce the running time of your algorithm to $O(|V|)$?

- Show that if an undirected graph with n vertices has k connected components, then it has at least $n - k$ edges.
- Consider an undirected graph $G = (V, E)$ with nonnegative edge weights $w_e \geq 0$. Suppose that you have computed a minimum spanning tree of G , and that you have also computed shortest paths to all nodes from a particular node $s \in V$.

Now suppose each edge weight is increased by 1: the new weights are $w'_e = w_e + 1$.

- Does the minimum spanning tree change? Give an example where it changes or prove it cannot change.
- Do the shortest paths change? Give an example where they change or prove they cannot change.

- Let $G = (V, E)$ be an undirected graph. Prove that if all its edge weights are distinct, then it has a unique minimum spanning tree.

- 5.7. Show how to find the *maximum* spanning tree of a graph, that is, the spanning tree of largest total weight.
- 5.8. Suppose you are given a weighted graph $G = (V, E)$ with a distinguished vertex s and where all edge weights are positive and distinct. Is it possible for a tree of shortest paths from s and a minimum spanning tree in G to not share any edges? If so, give an example. If not, give a reason.
- 5.9. The following statements may or may not be correct. In each case, either prove it (if it is correct) or give a counterexample (if it isn't correct). Always assume that the graph $G = (V, E)$ is undirected. Do not assume that edge weights are distinct unless this is specifically stated.
- (a) If graph G has more than $|V| - 1$ edges, and there is a unique heaviest edge, then this edge cannot be part of a minimum spanning tree.
 - (b) If G has a cycle with a unique heaviest edge e , then e cannot be part of any MST.
 - (c) Let e be any edge of minimum weight in G . Then e must be part of some MST.
 - (d) If the lightest edge in a graph is unique, then it must be part of every MST.
 - (e) If e is part of some MST of G , then it must be a lightest edge across some cut of G .
 - (f) If G has a cycle with a unique lightest edge e , then e must be part of every MST.
 - (g) The shortest-path tree computed by Dijkstra's algorithm is necessarily an MST.
 - (h) The shortest path between two nodes is necessarily part of some MST.
 - (i) Prim's algorithm works correctly when there are negative edges.
 - (j) (For any $r > 0$, define an r -path to be a path whose edges all have weight $< r$.) If G contains an r -path from node s to t , then every MST of G must also contain an r -path from node s to node t .
- 5.10. Let T be an MST of graph G . Given a connected subgraph H of G , show that $T \cap H$ is contained in some MST of H .
- 5.11. Give the state of the disjoint-sets data structure after the following sequence of operations, starting from singleton sets $\{1\}, \dots, \{8\}$. Use path compression. In case of ties, always make the lower numbered root point to the higher numbered one.
- union(1, 2), union(3, 4), union(5, 6), union(7, 8), union(1, 4), union(6, 7), union(4, 5), find(1)
- 5.12. Suppose you implement the disjoint-sets data structure using union-by-rank but not path compression. Give a sequence of m union and find operations on n elements that take $\Omega(m \log n)$ time.
- 5.13. A long string consists of the four characters A, C, G, T ; they appear with frequency 31%, 20%, 9%, and 40%, respectively. What is the Huffman encoding of these four characters?
- 5.14. Suppose the symbols a, b, c, d, e occur with frequencies $1/2, 1/4, 1/8, 1/16, 1/16$, respectively.
- (a) What is the Huffman encoding of the alphabet?
 - (b) If this encoding is applied to a file consisting of 1,000,000 characters with the given frequencies, what is the length of the encoded file in bits?
- 5.15. We use Huffman's algorithm to obtain an encoding of alphabet $\{a, b, c\}$ with frequencies f_a, f_b, f_c . In each of the following cases, either give an example of frequencies (f_a, f_b, f_c) that would yield the specified code, or explain why the code cannot possibly be obtained (no matter what the frequencies are).

- (a) Code: $\{0, 10, 11\}$
- (b) Code: $\{0, 1, 00\}$
- (c) Code: $\{10, 01, 00\}$

5.16. Prove the following two properties of the Huffman encoding scheme.

- (a) If some character occurs with frequency more than $2/5$, then there is guaranteed to be a codeword of length 1.
- (b) If all characters occur with frequency less than $1/3$, then there is guaranteed to be no codeword of length 1.

5.17. Under a Huffman encoding of n symbols with frequencies $f_1, f_2, \dots, f_n$, what is the longest a codeword could possibly be? Give an example set of frequencies that would produce this case.

5.18. The following table gives the frequencies of the letters of the English language (including the blank for separating words) in a particular corpus.

blank	18.3%	r	4.8%	y	1.6%
e	10.2%	d	3.5%	p	1.6%
t	7.7%	l	3.4%	b	1.3%
a	6.8%	c	2.6%	v	0.9%
o	5.9%	u	2.4%	k	0.6%
i	5.8%	m	2.1%	j	0.2%
n	5.5%	w	1.9%	x	0.2%
s	5.1%	f	1.8%	q	0.1%
h	4.9%	g	1.7%	z	0.1%

- (a) What is the optimum Huffman encoding of this alphabet?
- (b) What is the expected number of bits per letter?
- (c) Suppose now that we calculate the entropy of these frequencies

$$H = \sum_{i=1}^{26} p_i \log \frac{1}{p_i}$$

(see the box in page 150). Would you expect it to be larger or smaller than your answer above? Explain.

- (d) Do you think that this is the limit of how much English text can be compressed? What features of the English language, besides letters and their frequencies, should a better compression scheme take into account?

5.19. *Entropy*. Consider a distribution over n possible outcomes, with probabilities $p_1, p_2, \dots, p_n$.

- (a) Just for this part of the problem, assume that each p_i is a power of 2 (that is, of the form $1/2^k$). Suppose a long sequence of m samples is drawn from the distribution and that for all $1 \leq i \leq n$, the i^{th} outcome occurs exactly mp_i times in the sequence. Show that if Huffman encoding is applied to this sequence, the resulting encoding will have length

$$\sum_{i=1}^n mp_i \log \frac{1}{p_i}.$$

- (b) Now consider arbitrary distributions—that is, the probabilities p_i are not restricted to powers of 2. The most commonly used measure of the *amount of randomness* in the distribution is the *entropy*

$$\sum_{i=1}^n p_i \log \frac{1}{p_i}.$$

For what distribution (over n outcomes) is the entropy the largest possible? The smallest possible?

- 5.20. Give a linear-time algorithm that takes as input a tree and determines whether it has a *perfect matching*: a set of edges that touches each node exactly once.

A *feedback edge set* of an undirected graph $G = (V, E)$ is a subset of edges $E' \subseteq E$ that intersects every cycle of the graph. Thus, removing the edges E' will render the graph acyclic.

Give an efficient algorithm for the following problem:

Input: Undirected graph $G = (V, E)$ with positive edge weights w_e

Output: A feedback edge set $E' \subseteq E$ of minimum total weight $\sum_{e \in E'} w_e$

- 5.21. In this problem, we will develop a new algorithm for finding minimum spanning trees. It is based upon the following property:

Pick any cycle in the graph, and let e be the heaviest edge in that cycle. Then there is a minimum spanning tree that does not contain e .

- (a) Prove this property carefully.
 (b) Here is the new MST algorithm. The input is some undirected graph $G = (V, E)$ (in adjacency list format) with edge weights $\{w_e\}$.

```

sort the edges according to their weights
for each edge  $e \in E$ , in decreasing order of  $w_e$ :
    if  $e$  is part of a cycle of  $G$ :
         $G = G - e$  (that is, remove  $e$  from  $G$ )
return  $G$ 

```

Prove that this algorithm is correct.

- (c) On each iteration, the algorithm must check whether there is a cycle containing a specific edge e . Give a linear-time algorithm for this task, and justify its correctness.
 (d) What is the overall time taken by this algorithm, in terms of $|E|$? Explain your answer.
- 5.22. You are given a graph $G = (V, E)$ with positive edge weights, and a minimum spanning tree $T = (V, E')$ with respect to these weights; you may assume G and T are given as adjacency lists. Now suppose the weight of a particular edge $e \in E$ is modified from $w(e)$ to a new value $\hat{w}(e)$. You wish to quickly update the minimum spanning tree T to reflect this change, without recomputing the entire tree from scratch. There are four cases. In each case give a linear-time algorithm for updating the tree.
- $e \notin E'$ and $\hat{w}(e) > w(e)$.
 - $e \notin E'$ and $\hat{w}(e) < w(e)$.
 - $e \in E'$ and $\hat{w}(e) < w(e)$.
 - $e \in E'$ and $\hat{w}(e) > w(e)$.

- 5.23. Sometimes we want light spanning trees with certain special properties. Here's an example.

Input: Undirected graph $G = (V, E)$; edge weights w_e ; subset of vertices $U \subset V$

Output: The lightest spanning tree in which the nodes of U are *leaves* (there might be other leaves in this tree as well).

(The answer isn't necessarily a minimum spanning tree.)

Give an algorithm for this problem which runs in $O(|E| \log |V|)$ time. (*Hint:* When you remove nodes U from the optimal solution, what is left?)

- 5.24. A binary counter of unspecified length supports two operations: increment (which increases its value by one) and reset (which sets its value back to zero). Show that, starting from an initially zero counter, any sequence of n increment and reset operations takes time $O(n)$; that is, the amortized time per operation is $O(1)$.
- 5.25. Here's a problem that occurs in automatic program analysis. For a set of variables $x_1, \dots, x_n$, you are given some *equality* constraints, of the form " $x_i = x_j$ " and some *inequality* constraints, of the form " $x_i \neq x_j$." Is it possible to satisfy all of them?

For instance, the constraints

$$x_1 = x_2, x_2 = x_3, x_3 = x_4, x_1 \neq x_4$$

cannot be satisfied. Give an efficient algorithm that takes as input m constraints over n variables and decides whether the constraints can be satisfied.

- 5.26. *Graphs with prescribed degree sequences.* Given a list of n positive integers $d_1, d_2, \dots, d_n$, we want to efficiently determine whether there exists an undirected graph $G = (V, E)$ whose nodes have degrees precisely $d_1, d_2, \dots, d_n$. That is, if $V = \{v_1, \dots, v_n\}$, then the degree of v_i should be exactly d_i . We call $(d_1, \dots, d_n)$ the *degree sequence of G* . This graph G should not contain self-loops (edges with both endpoints equal to the same node) or multiple edges between the same pair of nodes.
- (a) Give an example of d_1, d_2, d_3, d_4 where all the $d_i \leq 3$ and $d_1 + d_2 + d_3 + d_4$ is even, but for which no graph with degree sequence (d_1, d_2, d_3, d_4) exists.
 - (b) Suppose that $d_1 \geq d_2 \geq \dots \geq d_n$ and that there exists a graph $G = (V, E)$ with degree sequence $(d_1, \dots, d_n)$. We want to show that there must exist a graph that has this degree sequence and where in addition the neighbors of v_1 are $v_2, v_3, \dots, v_{d_1+1}$. The idea is to gradually transform G into a graph with the desired additional property.
 - i. Suppose the neighbors of v_1 in G are not $v_2, v_3, \dots, v_{d_1+1}$. Show that there exists $i < j \leq n$ and $u \in V$ such that $\{v_1, v_i\}, \{u, v_j\} \notin E$ and $\{v_1, v_j\}, \{u, v_i\} \in E$.
 - ii. Specify the changes you would make to G to obtain a new graph $G' = (V, E')$ with the same degree sequence as G and where $(v_1, v_i) \in E'$.
 - iii. Now show that there must be a graph with the given degree sequence but in which v_1 has neighbors $v_2, v_3, \dots, v_{d_1+1}$.
 - (c) Using the result from part (b), describe an algorithm that on input $d_1, \dots, d_n$ (not necessarily sorted) decides whether there exists a graph with this degree sequence. Your algorithm should run in time polynomial in n and in $m = \sum_{i=1}^n d_i$.

- 5.27. Alice wants to throw a party and is deciding whom to call. She has n people to choose from, and she has made up a list of which pairs of these people know each other. She wants to pick as many people as possible, subject to two constraints: at the party, each person should have at least five other people whom they know *and* five other people whom they don't know.

Give an efficient algorithm that takes as input the list of n people and the list of pairs who know each other and outputs the best choice of party invitees. Give the running time in terms of n .

- 5.28. A *prefix-free encoding* of a finite alphabet Γ assigns each symbol in Γ a binary codeword, such that no codeword is a prefix of another codeword.

Show that such an encoding can be represented by a full binary tree in which each leaf corresponds to a unique element of Γ , whose codeword is generated by the path from the root to that leaf (interpreting a left branch as 0 and a right branch as 1).

- 5.29. *Ternary Huffman*. Trimedia Disks Inc. has developed “ternary” hard disks. Each cell on a disk can now store values 0, 1, or 2 (instead of just 0 or 1). To take advantage of this new technology, provide a modified Huffman algorithm for compressing sequences of characters from an alphabet of size n , where the characters occur with known frequencies $f_1, f_2, \dots, f_n$. Your algorithm should encode each character with a variable-length codeword over the values 0, 1, 2 such that no codeword is a prefix of another codeword and so as to obtain the maximum possible compression. Prove that your algorithm is correct.

- 5.30. The basic intuition behind Huffman’s algorithm, that frequent blocks should have short encodings and infrequent blocks should have long encodings, is also at work in English, where typical words like I, you, is, and, to, from, and so on are short, and rarely used words like velociraptor are longer.

However, words like fire!, help!, and run! are short not because they are frequent, but perhaps because time is precious in situations where they are used.

To make things theoretical, suppose we have a file composed of m different words, with frequencies $f_1, \dots, f_m$. Suppose also that for the i th word, the cost per bit of encoding is c_i . Thus, if we find a prefix-free code where the i th word has a codeword of length l_i , then the total cost of the encoding will be $\sum_i f_i \cdot c_i \cdot l_i$.

Show how to modify Huffman’s algorithm to find the prefix-free encoding of minimum total cost.

- 5.31. A server has n customers waiting to be served. The service time required by each customer is known in advance: it is t_i minutes for customer i . So if, for example, the customers are served in order of increasing i , then the i th customer has to wait $\sum_{j=1}^i t_j$ minutes.

We wish to minimize the total waiting time

$$T = \sum_{i=1}^n (\text{time spent waiting by customer } i).$$

Give an efficient algorithm for computing the optimal order in which to process the customers.

- 5.32. Show how to implement the stingy algorithm for Horn formula satisfiability (Section 5.3) in time that is linear in the length of the formula (the number of occurrences of literals in it). (*Hint*: Use a directed graph, with one node per variable, to represent the implications.)
- 5.33. Show that for any integer n that is a power of 2, there is an instance of the set cover problem (Section 5.4) with the following properties:

- i. There are n elements in the base set.
- ii. The optimal cover uses just two sets.
- iii. The greedy algorithm picks at least $\log n$ sets.

Thus the approximation ratio we derived in the chapter is tight.

Exercises

- 6.1. A *contiguous subsequence* of a list S is a subsequence made up of consecutive elements of S . For instance, if S is

$5, 15, -30, 10, -5, 40, 10,$

then $15, -30, 10$ is a contiguous subsequence but $5, 15, 40$ is not. Give a linear-time algorithm for the following task:

Input: A list of numbers, $a_1, a_2, \dots, a_n$.

Output: The contiguous subsequence of maximum sum (a subsequence of length zero has sum zero).

For the preceding example, the answer would be $10, -5, 40, 10$, with a sum of 55.

(*Hint:* For each $j \in \{1, 2, \dots, n\}$, consider contiguous subsequences ending exactly at position j .)

- 6.2. You are going on a long trip. You start on the road at mile post 0. Along the way there are n hotels, at mile posts $a_1 < a_2 < \dots < a_n$, where each a_i is measured from the starting point. The only places you are allowed to stop are at these hotels, but you can choose which of the hotels you stop at. You must stop at the final hotel (at distance a_n), which is your destination.

You'd ideally like to travel 200 miles a day, but this may not be possible (depending on the spacing of the hotels). If you travel x miles during a day, the *penalty* for that day is $(200 - x)^2$. You want to plan your trip so as to minimize the total penalty—that is, the sum, over all travel days, of the daily penalties.

Give an efficient algorithm that determines the optimal sequence of hotels at which to stop.

- 6.3. Yuckdonald's is considering opening a series of restaurants along Quaint Valley Highway (QVH). The n possible locations are along a straight line, and the distances of these locations from the start of QVH are, in miles and in increasing order, $m_1, m_2, \dots, m_n$. The constraints are as follows:
- At each location, Yuckdonald's may open at most one restaurant. The expected profit from opening a restaurant at location i is p_i , where $p_i > 0$ and $i = 1, 2, \dots, n$.
 - Any two restaurants should be at least k miles apart, where k is a positive integer.

Give an efficient algorithm to compute the maximum expected total profit subject to the given constraints.

- 6.4. You are given a string of n characters $s[1 \dots n]$, which you believe to be a corrupted text document in which all punctuation has vanished (so that it looks something like “itwasthebestoftimes...”). You wish to reconstruct the document using a dictionary, which is available in the form of a Boolean function $\text{dict}(\cdot)$: for any string w ,

$$\text{dict}(w) = \begin{cases} \text{true} & \text{if } w \text{ is a valid word} \\ \text{false} & \text{otherwise.} \end{cases}$$

- (a) Give a dynamic programming algorithm that determines whether the string $s[\cdot]$ can be reconstituted as a sequence of valid words. The running time should be at most $O(n^2)$, assuming calls to dict take unit time.
- (b) In the event that the string is valid, make your algorithm output the corresponding sequence of words.
- 6.5. *Pebbling a checkerboard.* We are given a checkerboard which has 4 rows and n columns, and has an integer written in each square. We are also given a set of $2n$ pebbles, and we want to place some or all of these on the checkerboard (each pebble can be placed on exactly one square) so as to maximize the sum of the integers in the squares that are covered by pebbles. There is one constraint: for a placement of pebbles to be legal, no two of them can be on horizontally or vertically adjacent squares (diagonal adjacency is fine).

- (a) Determine the number of legal *patterns* that can occur in any column (in isolation, ignoring the pebbles in adjacent columns) and describe these patterns.

Call two patterns *compatible* if they can be placed on adjacent columns to form a legal placement. Let us consider subproblems consisting of the first k columns $1 \leq k \leq n$. Each subproblem can be assigned a *type*, which is the pattern occurring in the last column.

- (b) Using the notions of compatibility and type, give an $O(n)$ -time dynamic programming algorithm for computing an optimal placement.
- 6.6. Let us define a multiplication operation on three symbols a, b, c according to the following table; thus $ab = b$, $ba = c$, and so on. Notice that the multiplication operation defined by the table is neither associative nor commutative.

	a	b	c
a	b	b	a
b	c	b	a
c	a	c	c

Find an efficient algorithm that examines a string of these symbols, say $bbbac$, and decides whether or not it is possible to parenthesize the string in such a way that the value of the resulting expression is a . For example, on input $bbbac$ your algorithm should return *yes* because $((b(bb))(ba))c = a$.

- 6.7. A subsequence is *palindromic* if it is the same whether read left to right or right to left. For instance, the sequence

$A, C, G, T, G, T, C, A, A, A, A, T, C, G$

has many palindromic subsequences, including A, C, G, C, A and A, A, A, A (on the other hand, the subsequence A, C, T is *not* palindromic). Devise an algorithm that takes a sequence $x[1 \dots n]$ and returns the (length of the) longest palindromic subsequence. Its running time should be $O(n^2)$.

- 6.8. Given two strings $x = x_1x_2 \cdots x_n$ and $y = y_1y_2 \cdots y_m$, we wish to find the length of their *longest common substring*, that is, the largest k for which there are indices i and j with $x_i x_{i+1} \cdots x_{i+k-1} = y_j y_{j+1} \cdots y_{j+k-1}$. Show how to do this in time $O(mn)$.
- 6.9. A certain string-processing language offers a primitive operation which splits a string into two pieces. Since this operation involves copying the original string, it takes n units of time for a string of length n , regardless of the location of the cut. Suppose, now, that you want to break a string into many pieces. The order in which the breaks are made can affect the total running time. For example, if you want to cut a 20-character string at positions 3 and 10, then making the first cut at position 3 incurs a total cost of $20 + 17 = 37$, while doing position 10 first has a better cost of $20 + 10 = 30$.
Give a dynamic programming algorithm that, given the locations of m cuts in a string of length n , finds the minimum cost of breaking the string into $m + 1$ pieces.
- 6.10. *Counting heads.* Given integers n and k , along with $p_1, \dots, p_n \in [0, 1]$, you want to determine the probability of obtaining exactly k heads when n biased coins are tossed independently at random, where p_i is the probability that the i th coin comes up heads. Give an $O(n^2)$ algorithm for this task.² Assume you can multiply and add two numbers in $[0, 1]$ in $O(1)$ time.
- 6.11. Given two strings $x = x_1x_2 \cdots x_n$ and $y = y_1y_2 \cdots y_m$, we wish to find the length of their *longest common subsequence*, that is, the largest k for which there are indices $i_1 < i_2 < \cdots < i_k$ and $j_1 < j_2 < \cdots < j_k$ with $x_{i_1} x_{i_2} \cdots x_{i_k} = y_{j_1} y_{j_2} \cdots y_{j_k}$. Show how to do this in time $O(mn)$.
- 6.12. You are given a convex polygon P on n vertices in the plane (specified by their x and y coordinates). A *triangulation* of P is a collection of $n - 3$ diagonals of P such that no two diagonals intersect (except possibly at their endpoints). Notice that a triangulation splits the polygon's interior into $n - 2$ disjoint triangles. The cost of a triangulation is the sum of the lengths of the diagonals in it. Give an efficient algorithm for finding a triangulation of minimum cost. (*Hint:* Label the vertices of P by $1, \dots, n$, starting from an arbitrary vertex and walking clockwise. For $1 \leq i < j \leq n$, let the subproblem $A(i, j)$ denote the minimum cost triangulation of the polygon spanned by vertices $i, i + 1, \dots, j$.)
- 6.13. Consider the following game. A “dealer” produces a sequence $s_1 \cdots s_n$ of “cards,” face up, where each card s_i has a value v_i . Then two players take turns picking a card from the sequence, but can only pick the first or the last card of the (remaining) sequence. The goal is to collect cards of largest total value. (For example, you can think of the cards as bills of different denominations.) Assume n is even.
- Show a sequence of cards such that it is not optimal for the first player to start by picking up the available card of larger value. That is, the natural *greedy* strategy is suboptimal.
 - Give an $O(n^2)$ algorithm to compute an optimal strategy for the first player. Given the initial sequence, your algorithm should precompute in $O(n^2)$ time some information, and then the first player should be able to make each move optimally in $O(1)$ time by looking up the precomputed information.
- 6.14. *Cutting cloth.* You are given a rectangular piece of cloth with dimensions $X \times Y$, where X and Y are positive integers, and a list of n products that can be made using the cloth. For each product $i \in [1, n]$ you know that a rectangle of cloth of dimensions $a_i \times b_i$ is needed and that the final selling price of the product is c_i . Assume the a_i , b_i , and c_i are all positive integers. You have a machine that can cut any rectangular piece of cloth into two pieces either horizontally or vertically. Design an algorithm that determines the best return on the $X \times Y$ piece of cloth, that is, a strategy for cutting the cloth so that the products made from the resulting pieces give the

²In fact, there is also a $O(n \log^2 n)$ algorithm within your reach.

maximum sum of selling prices. You are free to make as many copies of a given product as you wish, or none if desired.

- 6.15. Suppose two teams, A and B , are playing a match to see who is the first to win n games (for some particular n). We can suppose that A and B are equally competent, so each has a 50% chance of winning any particular game. Suppose they have already played $i + j$ games, of which A has won i and B has won j . Give an efficient algorithm to compute the probability that A will go on to win the match. For example, if $i = n - 1$ and $j = n - 3$ then the probability that A will win the match is $7/8$, since it must win any of the next three games.
- 6.16. The *garage sale problem* (courtesy of Professor Lofti Zadeh). On a given Sunday morning, there are n garage sales going on, $g_1, g_2, \dots, g_n$. For each garage sale g_j , you have an estimate of its value to you, v_j . For any two garage sales you have an estimate of the transportation cost d_{ij} of getting from g_i to g_j . You are also given the costs d_{0j} and d_{j0} of going between your home and each garage sale. You want to find a tour of a *subset* of the given garage sales, starting and ending at home, that maximizes your total benefit minus your total transportation costs. Give an algorithm that solves this problem in time $O(n^2 2^n)$. (*Hint*: This is closely related to the traveling salesman problem.)
- 6.17. Given an unlimited supply of coins of denominations $x_1, x_2, \dots, x_n$, we wish to make change for a value v ; that is, we wish to find a set of coins whose total value is v . This might not be possible: for instance, if the denominations are 5 and 10 then we can make change for 15 but not for 12. Give an $O(nv)$ dynamic-programming algorithm for the following problem.

Input: $x_1, \dots, x_n; v$.

Question: Is it possible to make change for v using coins of denominations $x_1, \dots, x_n$?

- 6.18. Consider the following variation on the change-making problem (Exercise 6.17): you are given denominations $x_1, x_2, \dots, x_n$, and you want to make change for a value v , but you are allowed to use each denomination *at most once*. For instance, if the denominations are 1, 5, 10, 20, then you can make change for $16 = 1 + 15$ and for $31 = 1 + 10 + 20$ but not for 40 (because you can't use 20 twice).

Input: Positive integers $x_1, x_2, \dots, x_n$; another integer v .

Output: Can you make change for v , using each denomination x_i at most once?

Show how to solve this problem in time $O(nv)$.

- 6.19. Here is yet another variation on the change-making problem (Exercise 6.17). Given an unlimited supply of coins of denominations $x_1, x_2, \dots, x_n$, we wish to make change for a value v using at most k coins; that is, we wish to find a set of $\leq k$ coins whose total value is v . This might not be possible: for instance, if the denominations are 5 and 10 and $k = 6$, then we can make change for 55 but not for 65. Give an efficient dynamic-programming algorithm for the following problem.

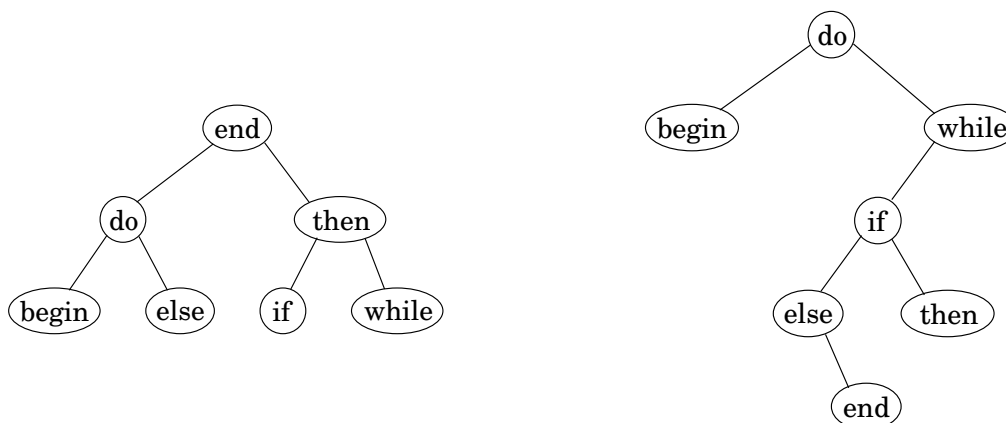
Input: $x_1, \dots, x_n; k; v$.

Question: Is it possible to make change for v using at most k coins, of denominations $x_1, \dots, x_n$?

- 6.20. *Optimal binary search trees*. Suppose we know the frequency with which keywords occur in programs of a certain language, for instance:

begin	5%
do	40%
else	8%

Figure 6.12 Two binary search trees for the keywords of a programming language.



end	4%
if	10%
then	10%
while	23%

We want to organize them in a *binary search tree*, so that the keyword in the root is alphabetically bigger than all the keywords in the left subtree and smaller than all the keywords in the right subtree (and this holds for all nodes).

Figure 6.12 has a nicely-balanced example on the left. In this case, when a keyword is being looked up, the number of comparisons needed is at most three: for instance, in finding “while”, only the three nodes “end”, “then”, and “while” get examined. But since we know the frequency with which keywords are accessed, we can use an even more fine-tuned cost function, the *average number of comparisons* to look up a word. For the search tree on the left, it is

$$\text{cost} = 1(0.04) + 2(0.40 + 0.10) + 3(0.05 + 0.08 + 0.10 + 0.23) = 2.42.$$

By this measure, the best search tree is the one on the right, which has a cost of 2.18.

Give an efficient algorithm for the following task.

Input: n words (in sorted order); frequencies of these words: $p_1, p_2, \dots, p_n$.

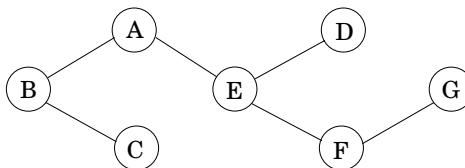
Output: The binary search tree of lowest cost (defined above as the expected number of comparisons in looking up a word).

- 6.21. A *vertex cover* of a graph $G = (V, E)$ is a subset of vertices $S \subseteq V$ that includes at least one endpoint of every edge in E . Give a linear-time algorithm for the following task.

Input: An undirected tree $T = (V, E)$.

Output: The size of the smallest vertex cover of T .

For instance, in the following tree, possible vertex covers include $\{A, B, C, D, E, F, G\}$ and $\{A, C, D, F\}$ but not $\{C, E, F\}$. The smallest vertex cover has size 3: $\{B, E, G\}$.



6.22. Give an $O(nt)$ algorithm for the following task.

Input: A list of n positive integers $a_1, a_2, \dots, a_n$; a positive integer t .

Question: Does some subset of the a_i 's add up to t ? (You can use each a_i at most once.)

(*Hint:* Look at subproblems of the form “does a subset of $\{a_1, a_2, \dots, a_i\}$ add up to s ?”)

6.23. A mission-critical production system has n stages that have to be performed sequentially; stage i is performed by machine M_i . Each machine M_i has a probability r_i of functioning reliably and a probability $1 - r_i$ of failing (and the failures are independent). Therefore, if we implement each stage with a single machine, the probability that the whole system works is $r_1 \cdot r_2 \cdots r_n$. To improve this probability we add redundancy, by having m_i copies of the machine M_i that performs stage i . The probability that all m_i copies fail simultaneously is only $(1 - r_i)^{m_i}$, so the probability that stage i is completed correctly is $1 - (1 - r_i)^{m_i}$ and the probability that the whole system works is $\prod_{i=1}^n (1 - (1 - r_i)^{m_i})$. Each machine M_i has a cost c_i , and there is a total budget B to buy machines. (Assume that B and c_i are positive integers.)

Given the probabilities $r_1, \dots, r_n$, the costs $c_1, \dots, c_n$, and the budget B , find the redundancies $m_1, \dots, m_n$ that are within the available budget and that maximize the probability that the system works correctly.

6.24. *Time and space complexity of dynamic programming.* Our dynamic programming algorithm for computing the edit distance between strings of length m and n creates a table of size $n \times m$ and therefore needs $O(mn)$ time and space. In practice, it will run out of space long before it runs out of time. How can this space requirement be reduced?

(a) Show that if we just want to compute the value of the edit distance (rather than the optimal sequence of edits), then only $O(n)$ space is needed, because only a small portion of the table needs to be maintained at any given time.

(b) Now suppose that we also want the optimal sequence of edits. As we saw earlier, this problem can be recast in terms of a corresponding grid-shaped dag, in which the goal is to find the optimal path from node $(0, 0)$ to node (n, m) . It will be convenient to work with this formulation, and while we're talking about convenience, we might as well also assume that m is a power of 2.

Let's start with a small addition to the edit distance algorithm that will turn out to be very useful. The optimal path in the dag must pass through an intermediate node $(k, m/2)$ for some k ; show how the algorithm can be modified to also return this value k .

(c) Now consider a recursive scheme:

```

procedure find-path( $(0, 0) \rightarrow (n, m)$ )
  compute the value  $k$  above
  find-path( $(0, 0) \rightarrow (k, m/2)$ )
  find-path( $(k, m/2) \rightarrow (n, m)$ )
  concatenate these two paths, with  $k$  in the middle
  
```

Show that this scheme can be made to run in $O(mn)$ time and $O(n)$ space.

6.25. Consider the following 3-PARTITION problem. Given integers $a_1, \dots, a_n$, we want to determine whether it is possible to partition of $\{1, \dots, n\}$ into three disjoint subsets I, J, K such that

$$\sum_{i \in I} a_i = \sum_{j \in J} a_j = \sum_{k \in K} a_k = \frac{1}{3} \sum_{i=1}^n a_i$$

For example, for input $(1, 2, 3, 4, 4, 5, 8)$ the answer is *yes*, because there is the partition $(1, 8)$, $(4, 5)$, $(2, 3, 4)$. On the other hand, for input $(2, 2, 3, 5)$ the answer is *no*.

Devise and analyze a dynamic programming algorithm for 3-PARTITION that runs in time polynomial in n and in $\sum_i a_i$.

- 6.26. *Sequence alignment.* When a new gene is discovered, a standard approach to understanding its function is to look through a database of known genes and find close matches. The closeness of two genes is measured by the extent to which they are *aligned*. To formalize this, think of a gene as being a long string over an alphabet $\Sigma = \{A, C, G, T\}$. Consider two genes (strings) $x = ATGCC$ and $y = TACGCA$. An alignment of x and y is a way of matching up these two strings by writing them in columns, for instance:

–	A	T	–	G	C	C
T	A	–	C	G	C	A

Here the “–” indicates a “gap.” The characters of each string must appear in order, and each column must contain a character from at least one of the strings. The score of an alignment is specified by a scoring matrix δ of size $(|\Sigma| + 1) \times (|\Sigma| + 1)$, where the extra row and column are to accommodate gaps. For instance the preceding alignment has the following score:

$$\delta(-, T) + \delta(A, A) + \delta(T, -) + \delta(-, C) + \delta(G, G) + \delta(C, C) + \delta(C, A).$$

Give a dynamic programming algorithm that takes as input two strings $x[1 \dots n]$ and $y[1 \dots m]$ and a scoring matrix δ , and returns the highest-scoring alignment. The running time should be $O(mn)$.

- 6.27. *Alignment with gap penalties.* The alignment algorithm of Exercise 6.26 helps to identify DNA sequences that are close to one another. The discrepancies between these closely matched sequences are often caused by errors in DNA replication. However, a closer look at the biological replication process reveals that the scoring function we considered earlier has a qualitative problem: nature often inserts or removes entire substrings of nucleotides (creating long gaps), rather than editing just one position at a time. Therefore, the penalty for a gap of length 10 should not be 10 times the penalty for a gap of length 1, but something significantly smaller.

Repeat Exercise 6.26, but this time use a modified scoring function in which the penalty for a gap of length k is $c_0 + c_1 k$, where c_0 and c_1 are given constants (and c_0 is larger than c_1).

- 6.28. *Local sequence alignment.* Often two DNA sequences are significantly different, but contain regions that are very similar and are *highly conserved*. Design an algorithm that takes an input two strings $x[1 \dots n]$ and $y[1 \dots m]$ and a scoring matrix δ (as defined in Exercise 6.26), and outputs substrings x' and y' of x and y , respectively, that have the highest-scoring alignment over all pairs of such substrings. Your algorithm should take time $O(mn)$.

- 6.29. *Exon chaining.* Each gene corresponds to a subregion of the overall genome (the DNA sequence); however, part of this region might be “junk DNA.” Frequently, a gene consists of several pieces called exons, which are separated by junk fragments called introns. This complicates the process of identifying genes in a newly sequenced genome.

Suppose we have a new DNA sequence and we want to check whether a certain gene (a string) is present in it. Because we cannot hope that the gene will be a contiguous subsequence, we look for partial matches—fragments of the DNA that are also present in the gene (actually, even these partial matches will be approximate, not perfect). We then attempt to assemble these fragments.

Let $x[1 \dots n]$ denote the DNA sequence. Each partial match can be represented by a triple (l_i, r_i, w_i) , where $x[l_i \dots r_i]$ is the fragment and w_i is a weight representing the strength of the match (it might be a local alignment score or some other statistical quantity). Many of these potential matches could be false, so the goal is to find a subset of the triples that are consistent (nonoverlapping) and have a maximum total weight.

Show how to do this efficiently.

6.30. *Reconstructing evolutionary trees by maximum parsimony.* Suppose we manage to sequence a particular gene across a whole bunch of different species. For concreteness, say there are n species, and the sequences are strings of length k over alphabet $\Sigma = \{A, C, G, T\}$. How can we use this information to reconstruct the evolutionary history of these species?

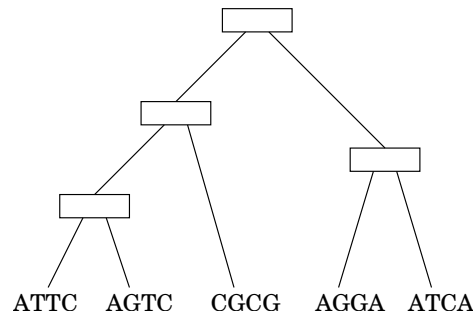
Evolutionary history is commonly represented by a tree whose leaves are the different species, whose root is their common ancestor, and whose internal branches represent speciation events (that is, moments when a new species broke off from an existing one). Thus we need to find the following:

- An evolutionary tree with the given species at the leaves.
- For each internal node, a string of length k : the gene sequence for that particular ancestor.

For each possible tree T , annotated with sequences $s(u) \in \Sigma^k$ at each of its nodes u , we can assign a score based on the principle of *parsimony*: fewer mutations are more likely.

$$\text{score}(T) = \sum_{(u,v) \in E(T)} (\text{number of positions on which } s(u) \text{ and } s(v) \text{ disagree}).$$

Finding the highest-score tree is a difficult problem. Here we will consider just a small part of it: suppose we know the structure of the tree, and we want to fill in the sequences $s(u)$ of the internal nodes u . Here's an example with $k = 4$ and $n = 5$:



- In this particular example, there are several maximum parsimony reconstructions of the internal node sequences. Find one of them.
- Give an efficient (in terms of n and k) algorithm for this task. (*Hint:* Even though the sequences might be long, you can do just one position at a time.)

Exercises

7.1. Consider the following linear program.

$$\begin{aligned} \text{maximize } & 5x + 3y \\ & 5x - 2y \geq 0 \\ & x + y \leq 7 \\ & x \leq 5 \\ & x \geq 0 \\ & y \geq 0 \end{aligned}$$

Plot the feasible region and identify the optimal solution.

7.2. Duckwheat is produced in Kansas and Mexico and consumed in New York and California. Kansas produces 15 shnupells of duckwheat and Mexico 8. Meanwhile, New York consumes 10 shnupells and California 13. The transportation costs per shnupell are \$4 from Mexico to New York, \$1 from Mexico to California, \$2 from Kansas to New York, and \$3 and from Kansas to California.

Write a linear program that decides the amounts of duckwheat (in shnupells and fractions of a shnupell) to be transported from each producer to each consumer, so as to minimize the overall transportation cost.

7.3. A cargo plane can carry a maximum weight of 100 tons and a maximum volume of 60 cubic meters. There are three materials to be transported, and the cargo company may choose to carry any amount of each, upto the maximum available limits given below.

- Material 1 has density 2 tons/cubic meter, maximum available amount 40 cubic meters, and revenue \$1,000 per cubic meter.
- Material 2 has density 1 ton/cubic meter, maximum available amount 30 cubic meters, and revenue \$1,200 per cubic meter.
- Material 3 has density 3 tons/cubic meter, maximum available amount 20 cubic meters, and revenue \$12,000 per cubic meter.

Write a linear program that optimizes revenue within the constraints.

7.4. Moe is deciding how much Regular Duff beer and how much Duff Strong beer to order each week. Regular Duff costs Moe \$1 per pint and he sells it at \$2 per pint; Duff Strong costs Moe \$1.50 per pint and he sells it at \$3 per pint. However, as part of a complicated marketing scam, the Duff company will only sell a pint of Duff Strong for each two pints or more of Regular Duff that Moe buys. Furthermore, due to past events that are better left untold, Duff will not sell Moe more than 3,000 pints per week. Moe knows that he can sell however much beer he has. Formulate a linear program for deciding how much Regular Duff and how much Duff Strong to buy, so as to maximize Moe's profit. Solve the program geometrically.

7.5. The Canine Products company offers two dog foods, Frisky Pup and Husky Hound, that are made from a blend of cereal and meat. A package of Frisky Pup requires 1 pound of cereal and 1.5 pounds of meat, and sells for \$7. A package of Husky Hound uses 2 pounds of cereal and 1 pound of meat, and sells for \$6. Raw cereal costs \$1 per pound and raw meat costs \$2 per pound. It also costs \$1.40 to package the Frisky Pup and \$0.60 to package the Husky Hound. A total of 240,000 pounds of cereal and 180,000 pounds of meat are available each month. The only production bottleneck is that the factory can only package 110,000 bags of Frisky Pup per month. Needless to say, management would like to maximize profit.

(a) Formulate the problem as a linear program in two variables.

(b) Graph the feasible region, give the coordinates of every vertex, and circle the vertex maximizing profit. What is the maximum profit possible?

7.6. Give an example of a linear program in two variables whose feasible region is infinite, but such that there is an optimum solution of bounded cost.

7.7. Find necessary and sufficient conditions on the reals a and b under which the linear program

$$\begin{aligned} \max \quad & x + y \\ \text{s.t.} \quad & ax + by \leq 1 \\ & x, y \geq 0 \end{aligned}$$

- (a) Is infeasible.
- (b) Is unbounded.
- (c) Has a unique optimal solution.

7.8. You are given the following points in the plane:

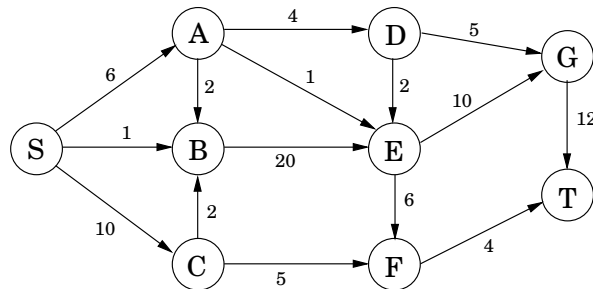
$$(1, 3), (2, 5), (3, 7), (5, 11), (7, 14), (8, 15), (10, 19).$$

You want to find a line $ax + by = c$ that approximately passes through these points (no line is a perfect fit). Write a linear program (you don't need to solve it) to find the line that minimizes the *maximum absolute error*,

$$\max_{1 \leq i \leq 7} |ax_i + by_i - c|.$$

7.9. A *quadratic programming* problem seeks to maximize a quadratic objective function (with terms like $3x_1^2$ or $5x_1x_2$) subject to a set of linear constraints. Give an example of a quadratic program in two variables x_1, x_2 such that the feasible region is nonempty and bounded, and yet none of the vertices of this region optimize the (quadratic) objective.

7.10. For the following network, with edge capacities as shown, find the maximum flow from S to T , along with a matching cut.



7.11. Write the dual to the following linear program.

$$\begin{aligned} \max \quad & x + y \\ \text{s.t.} \quad & 2x + y \leq 3 \\ & x + 3y \leq 5 \\ & x, y \geq 0 \end{aligned}$$

Find the optimal solutions to both primal and dual LPs.

7.12. For the linear program

$$\begin{aligned} \max \quad & x_1 - 2x_3 \\ \text{s.t.} \quad & x_1 - x_2 \leq 1 \\ & 2x_2 - x_3 \leq 1 \\ & x_1, x_2, x_3 \geq 0 \end{aligned}$$

prove that the solution $(x_1, x_2, x_3) = (3/2, 1/2, 0)$ is optimal.

7.13. *Matching pennies*. In this simple two-player game, the players (call them R and C) each choose an outcome, *heads* or *tails*. If both outcomes are equal, C gives a dollar to R ; if the outcomes are different, R gives a dollar to C .

(a) Represent the payoffs by a 2×2 matrix.

(b) What is the value of this game, and what are the optimal strategies for the two players?

7.14. The pizza business in Little Town is split between two rivals, Tony and Joey. They are each investigating strategies to steal business away from the other. Joey is considering either lowering prices or cutting bigger slices. Tony is looking into starting up a line of gourmet pizzas, or offering outdoor seating, or giving free sodas at lunchtime. The effects of these various strategies are summarized in the following payoff matrix (entries are dozens of pizzas, Joey's gain and Tony's loss).

		TONY		
		Gourmet	Seating	Free soda
JOEY	Lower price	+2	0	-3
	Bigger slices	-1	-2	+1

For instance, if Joey reduces prices and Tony goes with the gourmet option, then Tony will lose 2 dozen pizzas worth of business to Joey.

What is the value of this game, and what are the optimal strategies for Tony and Joey?

7.15. Find the value of the game specified by the following payoff matrix.

0	0	-1	-1
0	1	-2	-1
-1	-1	1	1
-1	0	0	1
1	-2	0	-3
1	-1	-1	-1
0	-3	2	-1
0	-2	1	-1

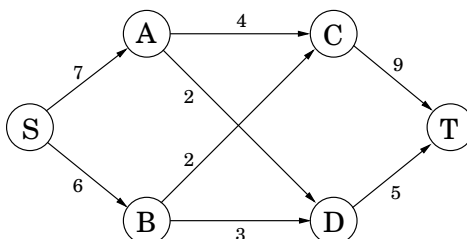
(Hint: Consider the mixed strategies $(1/3, 0, 0, 1/2, 1/6, 0, 0, 0)$ and $(2/3, 0, 0, 1/3)$.)

7.16. A salad is any combination of the following ingredients: (1) tomato, (2) lettuce, (3) spinach, (4) carrot, and (5) oil. Each salad must contain: (A) at least 15 grams of protein, (B) at least 2 and at most 6 grams of fat, (C) at least 4 grams of carbohydrates, (D) at most 100 milligrams of sodium. Furthermore, (E) you do not want your salad to be more than 50% greens by mass. The nutritional contents of these ingredients (per 100 grams) are

ingredient	energy (kcal)	protein (grams)	fat (grams)	carbohydrate (grams)	sodium (milligrams)
tomato	21	0.85	0.33	4.64	9.00
lettuce	16	1.62	0.20	2.37	8.00
spinach	371	12.78	1.58	74.69	7.00
carrot	346	8.39	1.39	80.70	508.20
oil	884	0.00	100.00	0.00	0.00

Find a linear programming applet on the Web and use it to make the salad with the fewest calories under the nutritional constraints. Describe your linear programming formulation and the optimal solution (the quantity of each ingredient and the value). Cite the Web resources that you used.

7.17. Consider the following network (the numbers are edge capacities).



- Find the maximum flow f and a minimum cut.
- Draw the residual graph G_f (along with its edge capacities). In this residual network, mark the vertices reachable from S and the vertices from which T is reachable.
- An edge of a network is called a *bottleneck edge* if increasing its capacity results in an increase in the maximum flow. List all bottleneck edges in the above network.
- Give a very simple example (containing at most four nodes) of a network which has no bottleneck edges.
- Give an efficient algorithm to identify all bottleneck edges in a network. (*Hint*: Start by running the usual network flow algorithm, and then examine the residual graph.)

7.18. There are many common variations of the maximum flow problem. Here are four of them.

- There are many sources and many sinks, and we wish to maximize the total flow from all sources to all sinks.
- Each *vertex* also has a capacity on the maximum flow that can enter it.
- Each edge has not only a capacity, but also a *lower bound* on the flow it must carry.
- The outgoing flow from each node u is not the same as the incoming flow, but is smaller by a factor of $(1 - \epsilon_u)$, where ϵ_u is a loss coefficient associated with node u .

Each of these can be solved efficiently. Show this by reducing (a) and (b) to the original max-flow problem, and reducing (c) and (d) to linear programming.

7.19. Suppose someone presents you with a solution to a max-flow problem on some network. Give a *linear* time algorithm to determine whether the solution does indeed give a maximum flow.

7.20. Consider the following generalization of the maximum flow problem.

You are given a directed network $G = (V, E)$ with edge capacities $\{c_e\}$. Instead of a single (s, t) pair, you are given multiple pairs $(s_1, t_1), (s_2, t_2), \dots, (s_k, t_k)$, where the s_i are sources of G and the t_i are sinks of G . You are also given k demands $d_1, \dots, d_k$. The goal is to find k flows $f^{(1)}, \dots, f^{(k)}$ with the following properties:

- $f^{(i)}$ is a valid flow from s_i to t_i .
- For each edge e , the total flow $f_e^{(1)} + f_e^{(2)} + \cdots + f_e^{(k)}$ does not exceed the capacity c_e .
- The size of each flow $f^{(i)}$ is at least the demand d_i .
- The size of the *total* flow (the sum of the flows) is as large as possible.

How would you solve this problem?

- 7.21. An edge of a flow network is called *critical* if decreasing the capacity of this edge results in a decrease in the maximum flow. Give an efficient algorithm that finds a critical edge in a network.
- 7.22. In a particular network $G = (V, E)$ whose edges have integer capacities c_e , we have already found the maximum flow f from node s to node t . However, we now find out that one of the capacity values we used was wrong: for edge (u, v) we used c_{uv} whereas it should have been $c_{uv} - 1$. This is unfortunate because the flow f uses that particular edge at full capacity: $f_{uv} = c_{uv}$. We could redo the flow computation from scratch, but there's a faster way. Show how a new optimal flow can be computed in $O(|V| + |E|)$ time.

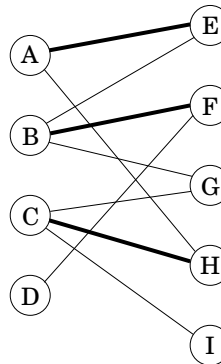
- 7.23. A *vertex cover* of an undirected graph $G = (V, E)$ is a subset of the vertices which touches every edge—that is, a subset $S \subset V$ such that for each edge $\{u, v\} \in E$, one or both of u, v are in S .

Show that the problem of finding the minimum vertex cover in a *bipartite* graph reduces to maximum flow. (*Hint*: Can you relate this problem to the minimum cut in an appropriate network?)

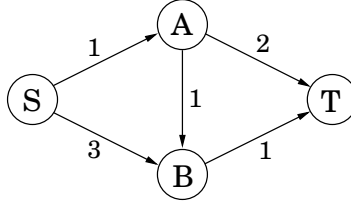
- 7.24. *Direct bipartite matching*. We've seen how to find a maximum matching in a bipartite graph via reduction to the maximum flow problem. We now develop a direct algorithm.

Let $G = (V_1 \cup V_2, E)$ be a bipartite graph (so each edge has one endpoint in V_1 and one endpoint in V_2), and let $M \subseteq E$ be a matching in the graph (that is, a set of edges that don't touch). A vertex is said to be *covered* by M if it is the endpoint of one of the edges in M . An *alternating path* is a path of odd length that starts and ends with a non-covered vertex, and whose edges alternate between M and $E - M$.

- (a) In the bipartite graph below, a matching M is shown in bold. Find an alternating path.



- (b) Prove that a matching M is maximal if and only if there does not exist an alternating path with respect to it.
- (c) Design an algorithm that finds an alternating path in $O(|V| + |E|)$ time using a variant of breadth-first search.
- (d) Give a direct $O(|V| \cdot |E|)$ algorithm for finding a maximal matching in a bipartite graph.
- 7.25. *The dual of maximum flow*. Consider the following network with edge capacities.



- Write the problem of finding the maximum flow from S to T as a linear program.
- Write down the dual of this linear program. There should be a dual variable for each edge of the network and for each vertex other than S, T .

Now we'll solve the same problem in full generality. Recall the linear program for a general maximum flow problem (Section 7.2).

- Write down the dual of this general flow LP, using a variable y_e for each edge and x_u for each vertex $u \neq s, t$.
- Show that any solution to the general dual LP must satisfy the following property: for any directed path from s to t in the network, the sum of the y_e values along the path must be at least 1.
- What are the intuitive meanings of the dual variables? Show that any $s - t$ cut in the network can be translated into a dual feasible solution whose cost is exactly the capacity of that cut.

7.26. In a satisfiable system of linear inequalities

$$\begin{array}{rcl}
 a_{11}x_1 + \cdots + a_{1n}x_n & \leq & b_1 \\
 & \vdots & \\
 a_{m1}x_1 + \cdots + a_{mn}x_n & \leq & b_m
 \end{array}$$

we describe the j th inequality as *forced-equal* if it is satisfied with equality by *every* solution $\mathbf{x} = (x_1, \dots, x_n)$ of the system. Equivalently, $\sum_i a_{ji}x_i \leq b_j$ is *not* forced-equal if there exists an $\mathbf{x}$ that satisfies the whole system and such that $\sum_i a_{ji}x_i < b_j$.

For example, in

$$\begin{array}{rcl}
 x_1 + x_2 & \leq & 2 \\
 -x_1 - x_2 & \leq & -2 \\
 x_1 & \leq & 1 \\
 -x_2 & \leq & 0
 \end{array}$$

the first two inequalities are forced-equal, while the third and fourth are not. A solution $\mathbf{x}$ to the system is called *characteristic* if, for every inequality I that is not forced-equal, $\mathbf{x}$ satisfies I without equality. In the instance above, such a solution is $(x_1, x_2) = (-1, 3)$, for which $x_1 < 1$ and $-x_2 < 0$ while $x_1 + x_2 = 2$ and $-x_1 - x_2 = -2$.

- Show that any satisfiable system has a characteristic solution.
- Given a satisfiable system of linear inequalities, show how to use linear programming to determine which inequalities are forced-equal, and to find a characteristic solution.

7.27. Show that the *change-making problem* (Exercise 6.17) can be formulated as an integer linear program. Can we solve this program as an LP, in the certainty that the solution will turn out to be integral (as in the case of bipartite matching)? Either prove it or give a counterexample.

7.28. *A linear program for shortest path.* Suppose we want to compute the shortest path from node s to node t in a directed graph with edge lengths $l_e > 0$.

- (a) Show that this is equivalent to finding an $s - t$ flow f that minimizes $\sum_e l_e f_e$ subject to $\text{size}(f) = 1$. There are no capacity constraints.
- (b) Write the shortest path problem as a linear program.
- (c) Show that the dual LP can be written as

$$\begin{aligned} \max \quad & x_s - x_t \\ \text{subject to} \quad & x_u - x_v \leq l_{uv} \text{ for all } (u, v) \in E \end{aligned}$$

- (d) An interpretation for the dual is given in the box on page 210. Why isn't our dual LP identical to the one on that page?

7.29. *Hollywood.* A film producer is seeking actors and investors for his new movie. There are n available actors; actor i charges s_i dollars. For funding, there are m available investors. Investor j will provide p_j dollars, but only on the condition that certain actors $L_j \subseteq \{1, 2, \dots, n\}$ are included in the cast (*all* of these actors L_j must be chosen in order to receive funding from investor j).

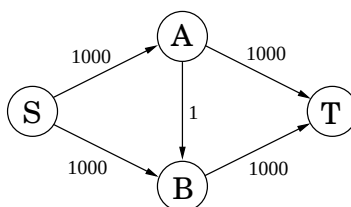
The producer's profit is the sum of the payments from investors minus the payments to actors. The goal is to maximize this profit.

- (a) Express this problem as an integer linear program in which the variables take on values $\{0, 1\}$.
- (b) Now relax this to a linear program, and show that there must in fact be an *integral* optimal solution (as is the case, for example, with maximum flow and bipartite matching).

7.30. *Hall's theorem.* Returning to the matchmaking scenario of Section 7.3, suppose we have a bipartite graph with boys on the left and an equal number of girls on the right. Hall's theorem says that there is a perfect matching if and only if the following condition holds: any subset S of boys is connected to at least $|S|$ girls.

Prove this theorem. (*Hint:* The max-flow min-cut theorem should be helpful.)

7.31. Consider the following simple network with edge capacities as shown.



- (a) Show that, if the Ford-Fulkerson algorithm is run on this graph, a careless choice of updates might cause it to take 1000 iterations. Imagine if the capacities were a million instead of 1000!

We will now find a strategy for choosing paths under which the algorithm is guaranteed to terminate in a reasonable number of iterations.

Consider an arbitrary directed network $(G = (V, E), s, t, \{c_e\})$ in which we want to find the maximum flow. Assume for simplicity that all edge capacities are at least 1, and define the capacity of an $s - t$ path to be the smallest capacity of its constituent edges. The *fattest path* from s to t is the path with the most capacity.

- (b) Show that the fattest $s - t$ path in a graph can be computed by a variant of Dijkstra's algorithm.
- (c) Show that the maximum flow in G is the sum of individual flows along at most $|E|$ paths from s to t .
- (d) Now show that if we always increase flow along the fattest path in the residual graph, then the Ford-Fulkerson algorithm will terminate in at most $O(|E| \log F)$ iterations, where F is the size of the maximum flow. (*Hint:* It might help to recall the proof for the greedy set cover algorithm in Section 5.4.)

In fact, an even simpler rule—finding a path in the residual graph using breadth-first search—guarantees that at most $O(|V| \cdot |E|)$ iterations will be needed.

Exercises

- 8.1. *Optimization versus search.* Recall the traveling salesman problem:

TSP

Input: A matrix of distances; a budget b

Output: A tour which passes through all the cities and has length $\leq b$, if such a tour exists.

The optimization version of this problem asks directly for the shortest tour.

TSP-OPT

Input: A matrix of distances

Output: The shortest tour which passes through all the cities.

Show that if TSP can be solved in polynomial time, then so can TSP-OPT.

- 8.2. *Search versus decision.* Suppose you have a procedure which runs in polynomial time and tells you whether or not a graph has a Rudrata path. Show that you can use it to develop a polynomial-time algorithm for RUDRATA PATH (which returns the actual path, if it exists).
- 8.3. STINGY SAT is the following problem: given a set of clauses (each a disjunction of literals) and an integer k , find a satisfying assignment in which at most k variables are true, if such an assignment exists. Prove that STINGY SAT is **NP**-complete.
- 8.4. Consider the CLIQUE problem restricted to graphs in which every vertex has degree at most 3. Call this problem CLIQUE-3.
- (a) Prove that CLIQUE-3 is in **NP**.
 - (b) What is wrong with the following proof of **NP**-completeness for CLIQUE-3?
We know that the CLIQUE problem in general graphs is **NP**-complete, so it is enough to present a reduction from CLIQUE-3 to CLIQUE. Given a graph G with vertices of degree ≤ 3 , and a parameter g , the reduction leaves the graph and the parameter unchanged: clearly the output of the reduction is a possible input for the CLIQUE problem. Furthermore, the answer to both problems is identical. This proves the correctness of the reduction and, therefore, the **NP**-completeness of CLIQUE-3.
 - (c) It is true that the VERTEX COVER problem remains **NP**-complete even when restricted to graphs in which every vertex has degree at most 3. Call this problem VC-3. What is wrong with the following proof of **NP**-completeness for CLIQUE-3?
We present a reduction from VC-3 to CLIQUE-3. Given a graph $G = (V, E)$ with node degrees bounded by 3, and a parameter b , we create an instance of CLIQUE-3 by leaving the graph unchanged and switching the parameter to $|V| - b$. Now, a subset $C \subseteq V$ is a vertex cover in G if and only if the complementary set $V - C$ is a clique in G . Therefore G has a vertex cover of size $\leq b$ if and only if it has a clique of size $\geq |V| - b$. This proves the correctness of the reduction and, consequently, the **NP**-completeness of CLIQUE-3.
 - (d) Describe an $O(|V|^4)$ algorithm for CLIQUE-3.
- 8.5. Give a simple reduction from 3D MATCHING to SAT, and another from RUDRATA CYCLE to SAT. (*Hint:* In the latter case you may use variables x_{ij} whose intuitive meaning is “vertex i is the j th vertex of the Hamilton cycle”; you then need to write clauses that express the constraints of the problem.)
- 8.6. On page 250 we saw that 3SAT remains **NP**-complete even when restricted to formulas in which each literal appears at most twice.

- (a) Show that if each literal appears at most *once*, then the problem is solvable in polynomial time.
 - (b) Show that INDEPENDENT SET remains **NP**-complete even in the special case when all the nodes in the graph have degree at most 4.
- 8.7. Consider a special case of 3SAT in which all clauses have exactly three literals, and each variable appears at most three times. Show that this problem can be solved in polynomial time. (*Hint*: create a bipartite graph with clauses on the left, variables on the right, and edges whenever a variable appears in a clause. Use Exercise 7.30 to show that this graph has a matching.)
- 8.8. In the EXACT 4SAT problem, the input is a set of clauses, each of which is a disjunction of exactly four literals, and such that each variable occurs at most once in each clause. The goal is to find a satisfying assignment, if one exists. Prove that EXACT 4SAT is **NP**-complete.
- 8.9. In the HITTING SET problem, we are given a family of sets $\{S_1, S_2, \dots, S_n\}$ and a budget b , and we wish to find a set H of size $\leq b$ which intersects every S_i , if such an H exists. In other words, we want $H \cap S_i \neq \emptyset$ for all i .
Show that HITTING SET is **NP**-complete.
- 8.10. *Proving **NP**-completeness by generalization.* For each of the problems below, prove that it is **NP**-complete by showing that it is a *generalization* of some **NP**-complete problem we have seen in this chapter.
- (a) SUBGRAPH ISOMORPHISM: Given as input two undirected graphs G and H , determine whether G is a subgraph of H (that is, whether by deleting certain vertices and edges of H we obtain a graph that is, up to renaming of vertices, identical to G), and if so, return the corresponding mapping of $V(G)$ into $V(H)$.
 - (b) LONGEST PATH: Given a graph G and an integer g , find in G a simple path of length g .
 - (c) MAX SAT: Given a CNF formula and an integer g , find a truth assignment that satisfies at least g clauses.
 - (d) DENSE SUBGRAPH: Given a graph and two integers a and b , find a set of a vertices of G such that there are at least b edges between them.
 - (e) SPARSE SUBGRAPH: Given a graph and two integers a and b , find a set of a vertices of G such that there are at most b edges between them.
 - (f) SET COVER. (This problem generalizes *two* known **NP**-complete problems.)
 - (g) RELIABLE NETWORK: We are given two $n \times n$ matrices, a distance matrix d_{ij} and a *connectivity requirement* matrix r_{ij} , as well as a budget b ; we must find a graph $G = (\{1, 2, \dots, n\}, E)$ such that (1) the total cost of all edges is b or less and (2) between any two distinct vertices i and j there are r_{ij} vertex-disjoint paths. (*Hint*: Suppose that all d_{ij} 's are 1 or 2, $b = n$, and all r_{ij} 's are 2. Which well known **NP**-complete problem is this?)
- 8.11. There are many variants of Rudrata's problem, depending on whether the graph is undirected or directed, and whether a cycle or path is sought. Reduce the DIRECTED RUDRATA PATH problem to each of the following.
- (a) The (undirected) RUDRATA PATH problem.
 - (b) The undirected RUDRATA (s, t) -PATH problem, which is just like RUDRATA PATH except that the endpoints of the path are specified in the input.
- 8.12. The k -SPANNING TREE problem is the following.

Input: An undirected graph $G = (V, E)$

Output: A spanning tree of G in which each node has degree $\leq k$, if such a tree exists.

Show that for any $k \geq 2$:

- (a) k -SPANNING TREE is a search problem.
- (b) k -SPANNING TREE is **NP**-complete. (*Hint:* Start with $k = 2$ and consider the relation between this problem and RUDRATA PATH.)

8.13. Determine which of the following problems are **NP**-complete and which are solvable in polynomial time. In each problem you are given an undirected graph $G = (V, E)$, along with:

- (a) A set of nodes $L \subseteq V$, and you must find a spanning tree such that its set of leaves includes the set L .
- (b) A set of nodes $L \subseteq V$, and you must find a spanning tree such that its set of leaves is precisely the set L .
- (c) A set of nodes $L \subseteq V$, and you must find a spanning tree such that its set of leaves is included in the set L .
- (d) An integer k , and you must find a spanning tree with k or fewer leaves.
- (e) An integer k , and you must find a spanning tree with k or more leaves.
- (f) An integer k , and you must find a spanning tree with exactly k leaves.

(*Hint:* All the **NP**-completeness proofs are by generalization, except for one.)

8.14. Prove that the following problem is **NP**-complete: given an undirected graph $G = (V, E)$ and an integer k , return a clique of size k as well as an independent set of size k , provided both exist.

8.15. Show that the following problem is **NP**-complete.

MAXIMUM COMMON SUBGRAPH

Input: Two graphs $G_1 = (V_1, E_1)$ and $G_2 = (V_2, E_2)$; a budget b .

Output: Two set of nodes $V'_1 \subseteq V_1$ and $V'_2 \subseteq V_2$ whose deletion leaves at least b nodes in each graph, and makes the two graphs identical.

8.16. We are feeling experimental and want to create a new dish. There are various ingredients we can choose from and we'd like to use as many of them as possible, but some ingredients don't go well with others. If there are n possible ingredients (numbered 1 to n), we write down an $n \times n$ matrix giving the *discord* between any pair of ingredients. This *discord* is a real number between 0.0 and 1.0, where 0.0 means "they go together perfectly" and 1.0 means "they really don't go together." Here's an example matrix when there are five possible ingredients.

	1	2	3	4	5
1	0.0	0.4	0.2	0.9	1.0
2	0.4	0.0	0.1	1.0	0.2
3	0.2	0.1	0.0	0.8	0.5
4	0.9	1.0	0.8	0.0	0.2
5	1.0	0.2	0.5	0.2	0.0

In this case, ingredients 2 and 3 go together pretty well whereas 1 and 5 clash badly. Notice that this matrix is necessarily symmetric; and that the diagonal entries are always 0.0. Any set of ingredients incurs a *penalty* which is the sum of all discord values between pairs of ingredients. For instance, the set of ingredients $\{1, 3, 5\}$ incurs a penalty of $0.2 + 1.0 + 0.5 = 1.7$. We want this penalty to be small.

EXPERIMENTAL CUISINE

Input: n , the number of ingredients to choose from; D , the $n \times n$ “discord” matrix; some number $p \geq 0$

Output: The maximum number of ingredients we can choose with penalty $\leq p$.

Show that if EXPERIMENTAL CUISINE is solvable in polynomial time, then so is 3SAT.

- 8.17. Show that for any problem Π in **NP**, there is an algorithm which solves Π in time $O(2^{p(n)})$, where n is the size of the input instance and $p(n)$ is a polynomial (which may depend on Π).
- 8.18. Show that if $\mathbf{P} = \mathbf{NP}$ then the RSA cryptosystem (Section 1.4.2) can be broken in polynomial time.

- 8.19. A *kite* is a graph on an even number of vertices, say $2n$, in which n of the vertices form a clique and the remaining n vertices are connected in a “tail” that consists of a path joined to one of the vertices of the clique. Given a graph and a goal g , the KITE problem asks for a subgraph which is a kite and which contains $2g$ nodes. Prove that KITE is **NP**-complete.

- 8.20. In an undirected graph $G = (V, E)$, we say $D \subseteq V$ is a *dominating set* if every $v \in V$ is either in D or adjacent to at least one member of D . In the DOMINATING SET problem, the input is a graph and a budget b , and the aim is to find a dominating set in the graph of size at most b , if one exists. Prove that this problem is **NP**-complete.

- 8.21. *Sequencing by hybridization.* One experimental procedure for identifying a new DNA sequence repeatedly probes it to determine which k -mers (substrings of length k) it contains. Based on these, the full sequence must then be reconstructed.

Let’s now formulate this as a combinatorial problem. For any string x (the DNA sequence), let $\Gamma(x)$ denote the multiset of all of its k -mers. In particular, $\Gamma(x)$ contains exactly $|x| - k + 1$ elements.

The reconstruction problem is now easy to state: given a multiset of k -length strings, find a string x such that $\Gamma(x)$ is exactly this multiset.

- (a) Show that the reconstruction problem reduces to RUDRATA PATH. (*Hint:* Construct a directed graph with one node for each k -mer, and with an edge from a to b if the last $k - 1$ characters of a match the first $k - 1$ characters of b .)
- (b) But in fact, there is much better news. Show that the same problem also reduces to EULER PATH. (*Hint:* This time, use one directed edge for each k -mer.)
- 8.22. In task scheduling, it is common to use a graph representation with a node for each task and a directed edge from task i to task j if i is a precondition for j . This directed graph depicts the precedence constraints in the scheduling problem. Clearly, a schedule is possible if and only if the graph is acyclic; if it isn’t, we’d like to identify the smallest number of constraints that must be dropped so as to make it acyclic.

Given a directed graph $G = (V, E)$, a subset $E' \subseteq E$ is called a *feedback arc set* if the removal of edges E' renders G acyclic.

FEEDBACK ARC SET (FAS): Given a directed graph $G = (V, E)$ and a budget b , find a feedback arc set of $\leq b$ edges, if one exists.

- (a) Show that FAS is in **NP**.

FAS can be shown to be **NP**-complete by a reduction from VERTEX COVER. Given an instance (G, b) of VERTEX COVER, where G is an undirected graph and we want a vertex cover of size $\leq b$, we construct a instance (G', b) of FAS as follows. If $G = (V, E)$ has n vertices $v_1, \dots, v_n$, then make $G' = (V', E')$ a directed graph with $2n$ vertices $w_1, w'_1, \dots, w_n, w'_n$, and $n + 2|E|$ (directed) edges:

- (w_i, w'_i) for all $i = 1, 2, \dots, n$.
 - (w'_i, w_j) and (w'_j, w_i) for every $(v_i, v_j) \in E$.
- (b) Show that if G contains a vertex cover of size b , then G' contains a feedback arc set of size b .
- (c) Show that if G' contains a feedback arc set of size b , then G contains a vertex cover of size (at most) b . (*Hint:* given a feedback arc set of size b in G' , you may need to first modify it slightly to obtain another one which is of a more convenient form, but is of the same size or smaller. Then, argue that G must contain a vertex cover of the same size as the modified feedback arc set.)
- 8.23. In the NODE-DISJOINT PATHS problem, the input is an undirected graph in which some vertices have been specially marked: a certain number of “sources” $s_1, s_2, \dots, s_k$ and an equal number of “destinations” $t_1, t_2, \dots, t_k$. The goal is to find k node-disjoint paths (that is, paths which have no nodes in common) where the i th path goes from s_i to t_i . Show that this problem is **NP**-complete. Here is a sequence of progressively stronger hints.
- (a) Reduce from 3SAT.
- (b) For a 3SAT formula with m clauses and n variables, use $k = m + n$ sources and destinations. Introduce one source/destination pair (s_x, t_x) for each variable x , and one source/destination pair (s_c, t_c) for each clause c .
- (c) For each 3SAT clause, introduce 6 new intermediate vertices, one for each literal occurring in that clause and one for its complement.
- (d) Notice that if the path from s_c to t_c goes through some intermediate vertex representing, say, an occurrence of variable x , then no other path can go through that vertex. What vertex would you like the other path to be forced to go through instead?

Exercises

- 9.1. In the backtracking algorithm for SAT, suppose that we always choose a subproblem (CNF formula) that has a clause that is as small as possible; and we expand it along a variable that appears in this small clause. Show that this is a polynomial-time algorithm in the special case in which the input formula has only clauses with two literals (that is, it is an instance of 2SAT).
- 9.2. Devise a backtracking algorithm for the RUDRATA PATH problem from a fixed vertex s . To fully specify such an algorithm you must define:
 - (a) What is a subproblem?
 - (b) How to choose a subproblem.
 - (c) How to expand a subproblem.

Argue briefly why your choices are reasonable.

- 9.3. Devise a branch-and-bound algorithm for the SET COVER problem. This entails deciding:
 - (a) What is a subproblem?
 - (b) How do you choose a subproblem to expand?
 - (c) How do you expand a subproblem?
 - (d) What is an appropriate lowerbound?

Do you think that your choices above will work well on typical instances of the problem? Why?

- 9.4. Given an undirected graph $G = (V, E)$ in which each node has degree $\leq d$, show how to efficiently find an independent set whose size is at least $1/(d+1)$ times that of the largest independent set.
- 9.5. *Local search for minimum spanning trees.* Consider the set of all spanning trees (not just minimum ones) of a weighted, connected, undirected graph $G = (V, E)$.

Recall from Section 5.1 that adding an edge e to a spanning tree T creates a unique cycle, and subsequently removing any other edge $e' \neq e$ from this cycle gives back a different spanning tree T' . We will say that T and T' differ by a single *edge swap* (e, e') and that they are *neighbors*.

- (a) Show that it is possible to move from any spanning tree T to any other spanning tree T' by performing a series of edge-swaps, that is, by moving from neighbor to neighbor. At most how many edge-swaps are needed?
- (b) Show that if T' is an MST, then it is possible to choose these swaps so that the costs of the spanning trees encountered along the way are nonincreasing. In other words, if the sequence of spanning trees encountered is

$$T = T_0 \rightarrow T_1 \rightarrow T_2 \rightarrow \cdots \rightarrow T_k = T',$$

then $\text{cost}(T_{i+1}) \leq \text{cost}(T_i)$ for all $i < k$.

- (c) Consider the following local search algorithm which is given as input an undirected graph G .

```

  Let  $T$  be any spanning tree of  $G$ 
  while there is an edge-swap  $(e, e')$  which reduces  $\text{cost}(T)$ :
     $T \leftarrow T + e - e'$ 
  return  $T$ 

```

Show that this procedure always returns a minimum spanning tree. At most how many iterations does it take?

- 9.6. In the MINIMUM STEINER TREE problem, the input consists of: a complete graph $G = (V, E)$ with distances d_{uv} between all pairs of nodes; and a distinguished set of *terminal nodes* $V' \subseteq V$. The goal is to find a minimum-cost tree that includes the vertices V' . This tree may or may not include nodes in $V - V'$.



Suppose the distances in the input are a metric (recall the definition on page 277). Show that an efficient ratio-2 approximation algorithm for MINIMUM STEINER TREE can be obtained by ignoring the nonterminal nodes and simply returning the minimum spanning tree on V' . (*Hint:* Recall our approximation algorithm for the TSP.)

- 9.7. In the MULTIWAY CUT problem, the input is an undirected graph $G = (V, E)$ and a set of terminal nodes $s_1, s_2, \dots, s_k \in V$. The goal is to find the minimum set of edges in E whose removal leaves all terminals in different components.
- (a) Show that this problem can be solved exactly in polynomial time when $k = 2$.
- (b) Give an approximation algorithm with ratio at most 2 for the case $k = 3$.
- (c) Design a local search algorithm for multiway cut.
- 9.8. In the MAX SAT problem, we are given a set of clauses, and we want to find an assignment that satisfies as many of them as possible.

- (a) Show that if this problem can be solved in polynomial time, then so can SAT.
- (b) Here's a very naive algorithm.

for each variable:

set its value to either 0 or 1 by flipping a coin

Suppose the input has m clauses, of which the j th has k_j literals. Show that the *expected* number of clauses satisfied by this simple algorithm is

$$\sum_{j=1}^m \left(1 - \frac{1}{2^{k_j}}\right) \geq \frac{m}{2}.$$

In other words, this is a 2-approximation in expectation! And if the clauses all contain k literals, then this approximation factor improves to $1 + 1/(2^k - 1)$.

- (c) Can you make this algorithm deterministic? (*Hint:* Instead of flipping a coin for each variable, select the value that satisfies the most of the as of yet unsatisfied clauses. What fraction of the clauses is satisfied in the end?)
- 9.9. In the MAXIMUM CUT problem we are given an undirected graph $G = (V, E)$ with a weight $w(e)$ on each edge, and we wish to separate the vertices into two sets S and $V - S$ so that the total weight of the edges between the two sets is as *large* as possible.
- For each $S \subseteq V$ define $w(S)$ to be the sum of all $w(e)$ over all edges $\{u, v\}$ such that $|S \cap \{u, v\}| = 1$. Obviously, MAX CUT is about maximizing $w(S)$ over all subsets of V .
- Consider the following local search algorithm for MAX CUT:

```

start with any  $S \subseteq V$ 
while there is a subset  $S' \subseteq V$  such that
   $||S'| - |S|| = 1$  and  $w(S') > w(S)$  do:
  set  $S = S'$ 

```

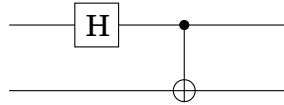
- (a) Show that this is an approximation algorithm for MAX CUT with ratio 2.
 - (b) But is it a polynomial-time algorithm?
- 9.10. Let us call a local search algorithm *exact* when it always produces the optimum solution. For example, the local search algorithm for the minimum spanning tree problem introduced in Problem 9.5 is exact. For another example, simplex can be considered an exact local search algorithm for linear programming.
- (a) Show that the 2-change local search algorithm for the TSP is not exact.
 - (b) Repeat for the $\lceil \frac{n}{2} \rceil$ -change local search algorithm, where n is the number of cities.
 - (c) Show that the $(n - 1)$ -change local search algorithm is exact.
 - (d) If A is an optimization problem, define A -IMPROVEMENT to be the following search problem: Given an instance x of A and a solution s of A , find another solution of x with better cost (or report that none exists, and thus s is optimum). For example, in TSP IMPROVEMENT we are given a distance matrix and a tour, and we are asked to find a better tour. It turns out that TSP IMPROVEMENT is **NP**-complete, and so is SET COVER IMPROVEMENT. (Can you prove this?)
 - (e) We say that a local search algorithm has *polynomial iteration* if each execution of the loop requires polynomial time. For example, the obvious implementations of the $(n - 1)$ -change local search algorithm for the TSP defined above do not have polynomial iteration. Show that, unless $\mathbf{P} = \mathbf{NP}$, there is no exact local search algorithm with polynomial iteration for the TSP and SET COVER problems.

Exercises

10.1. $|\psi\rangle = \frac{1}{\sqrt{2}}|00\rangle + \frac{1}{\sqrt{2}}|11\rangle$ is one of the famous “Bell states,” a highly entangled state of its two qubits. In this question we examine some of its strange properties.

- Suppose this Bell state could be decomposed as the (tensor) product of two qubits (recall the box on page 298), the first in state $\alpha_0|0\rangle + \alpha_1|1\rangle$ and the second in state $\beta_0|0\rangle + \beta_1|1\rangle$. Write four equations that the amplitudes $\alpha_0, \alpha_1, \beta_0$, and β_1 must satisfy. Conclude that the Bell state cannot be so decomposed.
- What is the result of measuring the first qubit of $|\psi\rangle$?
- What is the result of measuring the second qubit after measuring the first qubit?
- If the two qubits in state $|\psi\rangle$ are very far from each other, can you see why the answer to (c) is surprising?

10.2. Show that the following quantum circuit prepares the Bell state $|\psi\rangle = \frac{1}{\sqrt{2}}|00\rangle + \frac{1}{\sqrt{2}}|11\rangle$ on input $|00\rangle$: apply a Hadamard gate to the first qubit followed by a CNOT with the first qubit as the control and the second qubit as the target.



What does the circuit output on input 10, 01, and 11? These are the rest of the Bell basis states.

10.3. What is the quantum Fourier transform modulo M of the uniform superposition $\frac{1}{\sqrt{M}} \sum_{j=0}^{M-1} |j\rangle$?

10.4. What is the QFT modulo M of $|j\rangle$?

10.5. *Convolution-Multiplication.* Suppose we shift a superposition $|\alpha\rangle = \sum_j \alpha_j |j\rangle$ by l to get the superposition $|\alpha'\rangle = \sum_j \alpha_j |j+l\rangle$. If the QFT of $|\alpha\rangle$ is $|\beta\rangle$, show that the QFT of α' is β' , where $\beta'_j = \beta_j \omega^{lj}$. Conclude that if $|\alpha'\rangle = \sum_{j=0}^{M/k-1} \sqrt{\frac{k}{M}} |jk+l\rangle$, then $|\beta'\rangle = \frac{1}{\sqrt{k}} \sum_{j=0}^{k-1} \omega^{ljM/k} |jM/k\rangle$.

10.6. Show that if you apply the Hadamard gate to the inputs and outputs of a CNOT gate, the result is a CNOT gate with control and target qubits switched:



10.7. The CONTROLLED SWAP (C-SWAP) gate takes as input 3 qubits and swaps the second and third if and only if the first qubit is a 1.

- Show that each of the NOT, CNOT, and C-SWAP gates are their own inverses.
- Show how to implement an AND gate using a C-SWAP gate, i.e., what inputs a, b, c would you give to a C-SWAP gate so that one of the outputs is $a \wedge b$?
- How would you achieve fanout using just these three gates? That is, on input a and 0, output a and a .

- (d) Conclude therefore that for any classical circuit C there is an equivalent quantum circuit Q using just NOT and C-SWAP gates in the following sense: if C outputs y on input x , then Q outputs $|x, y, z\rangle$ on input $|x, 0, 0\rangle$. (Here z is some set of junk bits that are generated during this computation).
 - (e) Now show that that there is a quantum circuit Q^{-1} that outputs $|x, 0, 0\rangle$ on input $|x, y, z\rangle$.
 - (f) Show that there is a quantum circuit Q' made up of NOT, CNOT, and C-SWAP gates that outputs $|x, y, 0\rangle$ on input $|x, 0, 0\rangle$.
- 10.8. In this problem we will show that if $N = pq$ is the product of two odd primes, and if x is chosen uniformly at random between 0 and $N - 1$, such that $\gcd(x, N) = 1$, then with probability at least $3/8$, the order r of $x \bmod N$ is even, and moreover $x^{r/2}$ is a nontrivial square root of 1 mod N .
- (a) Let p be an odd prime and let x be a uniformly random number modulo p . Show that the order of $x \bmod p$ is even with probability at least $1/2$. (*Hint:* Use Fermat's little theorem (Section 1.3).)
 - (b) Use the Chinese remainder theorem (Exercise 1.37) to show that with probability at least $3/4$, the order r of $x \bmod N$ is even.
 - (c) If r is even, prove that the probability that $x^{r/2} \equiv \pm 1$ is at most $1/2$.